

Rapport de projet J3D

Echoes Of Lights

-

Les Debuggers

Sommaire :

1. Revue du TSD	3 - 6
1.1 Introduction	3
1.2 Synthèse Exécutive	3
1.3 Analyse par Domaine	3 - 6
1.4 Conclusion de la review du TSD	6 - 7
2. Rapports personnels	7 - 51
2.1. Antoine LAPP	7 - 13
2.2. Guillaume KLEIN	13 - 36
2.3. Antoine MUH	37 - 42
2.4. Gustave SCHWIEN	42 - 48
2.5. Valentin GOUSSOT	48 - 51
3. Récit de mise en œuvre du projet	52 - 56
3.1. Joies	52 - 53
3.2. Réussites / Succès	53 - 56
4. Annexes	56 - 66

1. Review du TSD :

1.1 Introduction

Echoes Of Lights est un jeu vidéo coopératif en 2D isométrique que nous avons développé en tant que groupe Les Debuggers. Les Debuggers sont constitués de Guillaume Klein, Antoine Lapp, Valentin Goussot, Antoine Muh et Gustave Schwien. Ce document présente notre progression complète à travers les trois soutenances.

Cette revue TSD finale documente comment nous avons transformé nos objectifs initiaux en un projet complet et fonctionnel, en montrant notre progression à travers chaque jalon du développement.

1.2 Synthèse Exécutive

TAUX D'ACCOMPLISSEMENT FINAL : 100%

Nous avons achevé Echoes Of Lights avec succès. Tous les domaines du TSD sont maintenant complètement finalisés et intégrés. Voir l'annexe 1.2.1

1.3. Analyse par Domaine

1.3.1 Pixel Art et Assets Graphiques

Progression : 20% → 50% → 100%

Nous avons débuté avec 20% des assets en place à la soutenance 1. Nous avons produit les tuiles de terrain et les premiers Spritesheets des joueurs, mais une grande partie de notre temps s'est concentrée sur le gameplay et les énigmes. À la soutenance 2, nous avons atteint 50% avec la majorité des éléments visuels essentiels complétés. Pour la version finale, nous avons finalisé tous les assets manquants : éléments de décor, ennemis, et effets visuels. Le résultat est une vision artistique complète et cohérente.

Réalisations Finales :

Nous avons créé un ensemble complet de tuiles de terrain en 64x64 pixels. Les Spritesheets ont été finalisés pour les deux personnages joueurs avec toutes les animations. Le design complet des ennemis (Slasher et Flinger) a été réalisé. Nous avons enrichi l'environnement avec des éléments de décor tels que les arbres, rochers et différents détails visuels.

1.3.2 Musique et Audio

Progression : 0% → 50% → 100%

La soutenance 1 montrait 0% car nous avons priorisé le gameplay. À la soutenance 2, nous avons intégré la musique principale composée par Valentin. Nous avons achevé cet aspect pour la version finale.

État Final :

Valentin a composé la musique du menu. Les musiques de gameplay fluides et immersives ont été composées par Gustave et Guillaume. Le système de gestion audio est complètement intégré avec un contrôle du volume complet.

1.3.3 Animation

Progression : 0% → 50% → 100%

Nous partions de zéro en termes d'animation à la soutenance 1. À la soutenance 2, nous avons créé la première version de l'animation d'attaque et les spritesheets de mouvement. La version finale présente des animations complètes et fluides. Ces animations ont été désignées et mises en Spritesheets par Antoine Muh.

Accomplissements :

Les Spritesheets incluent des mouvements dans 8 directions différentes. L'animation d'attaque de mêlée est fluide et impactante. Les animations des ennemis et de leurs interactions sont également complètement développées et synchronisées avec le gameplay.

1.3.4 Sound Design

Progression : 0% → 50% → 100%

Comme le pixel art, le sound design avait été mis de côté en priorité 1 pour le gameplay. À la soutenance 2, nous avons intégré 50% des effets sonores essentiels. Nous avons complété ce domaine pour la version finale. Cette tâche a été réalisée par Gustave.

Intégration Finale :

Les effets sonores d'attaque et d'impact sont maintenant présents. Les ambiances sonores pour chaque environnement ont été intégrées. Le feedback audio pour l'interface utilisateur et les énigmes complète l'expérience sonore du jeu.

1.3.5 Système Multijoueur

Progression : 90% → 100% → 100%

Le multijoueur était déjà à 90% dès la soutenance 1, ce qui témoigne de l'importance que nous avons accordée aux fondations techniques. Nous avons maintenu et affiné ce système tout au long du projet. Le multijoueur a été réalisé par Antoine Lapp.

Architecture P2P Stable :

La connexion directe entre deux joueurs fonctionne sans serveur central. La synchronisation fluide de l'état du jeu est garantie. La gestion robuste des actions partagées et des énigmes coopératives est complètement opérationnelle.

1.3.6 Gameplay et Mécanique

Progression : 50% → 90% → 100%

Le gameplay a évolué régulièrement. À la soutenance 1, nous avons 50% des mécaniques de base. À la soutenance 2, nous approchons de 100% avec l'attaque de mêlée et les énigmes majeures fonctionnelles. La version finale complète tous les systèmes.

Systèmes Finalisés :

L'attaque de mêlée est entièrement fonctionnelle avec une détection de coups précise. Deux compétences spécialisées sont disponibles pour chaque joueur. L'énigme des leviers est complètement opérationnelle. Le Jeu de la Vie (Game of Life) fonctionne comme deuxième énigme majeure.

1.3.7 Design de la Carte

Progression : 20% → 50% → 100%

Nous avons commencé avec une structure basique à la soutenance 1. À la soutenance 2, 50% était complétée avec tous les murs en place. Pour la version finale, nous avons ajouté tous les éléments de décor pour enrichir l'environnement.

État Complet :

L'ensemble complet des murs et délimitations est en place. Les salles sont bien définies et connectées de manière logique. Les éléments de décor enrichissent l'environnement pour créer une atmosphère immersive.

1.3.8 Déroulement du Jeu

Progression : 80% → 100% → 100%

La narration et le déroulement complet du jeu étaient définis dès le début. À la soutenance 1, nous avons 80% finalisés sur papier. Nous avons achevé les détails à la soutenance 2 et maintenu cet état pour la version finale. La progression narrative est fluide et cohérente. La structure complète du jeu va du prologue à l'épilogue avec une arc narratif bien établi.

1.3.9 Système d'Affichage Isométrique

Progression : 80% → 100% → 100%

Le moteur isométrique était bien avancé à la soutenance 1 avec 80% de complétion. Nous l'avons finalisé à la soutenance 2 et l'avons maintenu stable pour la version finale. C'est une fondation technique solide. Le rendu isométrique est fluide et performant. La gestion des couches et de la profondeur est correctement implémentée. L'architecture modulaire permet d'ajouter ou de modifier facilement des éléments.

1.3.10 Système de Menus

Progression : 30% → 100% → 100%

Nous avons 30% du système de menus à la soutenance 1 avec le menu principal basique. À la soutenance 2, nous avons atteint 100% en complétant le menu pause et les options. La version finale

maintient cet état avec tous les éléments d'interface fonctionnels. Le menu principal inclut toutes les options nécessaires. Le menu pause est accessible en jeu. Le menu options permet de contrôler le volume et d'ajuster les paramètres.

1.3.11 Système d'Intelligence Artificielle

Progression : 30% → 70% → 100%

L'IA était à ses débuts à la soutenance 1 avec seulement 20-30% des comportements implémentés. À la soutenance 2, nous avons 70% du système fonctionnel. Pour la version finale, nous avons complété tous les comportements et optimisations.

Système Complet :

Le pathfinding A* est entièrement fonctionnel. Les mouvements adaptatifs en 8 directions sont implémentés. Les comportements intelligents basés sur la distance au joueur fonctionnent correctement. La différenciation par type d'ennemi (Slasher, Flinger) est bien établie avec des comportements et des portées d'attaque distincts.

1.3.12 Développement des Joueurs

Progression : 70% → 90% → 100%

Le développement des joueurs avait une bonne progression dès la soutenance 1 avec 70%. À la soutenance 2, nous étions à 90% avec les animations et interactions terminées. La version finale complète les deux compétences spécialisées.

Systèmes Complets :

Les animations de mouvement sont disponibles pour toutes les directions. L'attaque de mêlée possède un minutage et une détection correcte. Deux compétences uniques sont assignées à chaque joueur. Les interactions sont fluides et bien intégrées avec l'environnement.

1.4. Conclusion de la review du TSD

Parcours du Projet

Nous avons transformé une vision initiale en un jeu entièrement fonctionnel sur trois phases distinctes de développement. Chaque soutenance a marqué un progrès significatif avec des domaines complétés et intégrés progressivement. La progression montre notre capacité à prioriser efficacement, à nous adapter aux défis techniques, et à livrer un produit cohérent malgré la complexité du projet. Le multijoueur et le système isométrique ont formé des fondations solides sur lesquelles nous avons construit les éléments créatifs.

Points Forts

L'architecture P2P robuste dès le début du projet a permis une intégration fluide des autres systèmes. La progression constante et mesurable à travers les trois soutenances démontre une gestion de projet

efficace. La cohérence artistique et technique a été maintenue tout au long du développement. Le jeu final est entièrement jouable et agréable à expérimenter pour les utilisateurs.

Apprentissages

Nous avons confirmé l'importance de prioriser les fondations techniques avant les éléments visuels. La communication d'équipe s'est révélée cruciale malgré la distance entre les membres du groupe. La modularité du code a facilité les itérations et les ajustements pendant le développement. Le testing régulier a permis de prévenir les intégrations problématiques et d'identifier rapidement les bugs. Cette expérience nous sera très utile pour nos futurs projets de développement logiciel.

2. Rapports personnels

2.1. Antoine LAPP :

2.1.1 Introduction

Dans la création du jeu Echoes Of Lights, je me suis principalement consacré à la dimension réseau du jeu, c'est-à-dire de permettre aux joueurs ensemble au sein d'un même environnement. La partie multijoueur est un pilier fondamental de ce projet car l'importante majorité du gameplay, si ce n'est la totalité repose sur la collaboration entre les deux joueurs.

J'ai donc développé le multijoueur en tant que telle mais j'ai aussi aidé mes camarades à adapter leur code au multijoueur en partageant les diverses informations via le réseau notamment pour les joueurs et les ennemis.

J'ai également participé à la cohérence visuelle générale du jeu et l'interface utilisateur notamment en implémentant, le HUD informant le joueur sur sa vie et les capacités dont il dispose, mais aussi en créant un menu pause permettant au joueur de modifier différentes options et de gérer sa partie. J'ai aussi rendu le jeu plus immersif en ajoutant des transitions, en implémentant l'affichage en plein écran, en participant à l'intégration des animations désignées par Antoine Muh.

Au-delà de l'aspect visuel, j'ai aussi participé dans le développement de certains éléments des énigmes que les joueurs doivent surmonter. J'ai notamment créé le système de portes permettant de restreindre l'accès à certaines zones aux joueurs avant d'avoir résolu l'énigme. J'ai aussi créé les artefacts et gérer le changement de map une fois que les 4 ont été récupérés.

Je me suis aussi chargé de créer une bonne partie des tutoriels afin de rendre le jeu facilement compréhensibles par les utilisateurs, même ceux qui ne sont pas habitués à utiliser ce genre de produits. J'ai notamment fait l'affichage d'une check liste pour indiquer les différentes touches du clavier à utiliser. J'ai aussi créé des panneaux d'affichage qui s'intègrent pleinement dans l'environnement afin de donner au joueur des indications sur la résolution de l'énigme auquel il fait face sans pour autant lui donner directement la réponse.

Enfin, je me suis aussi occupé de compiler l'ensemble de notre code python en un unique exécutable. J'ai donc dû gérer les dépendances et les différents fichiers. En collaboration avec Guillaume j'ai aussi pu mettre en place la possibilité d'enregistrer et de charger des sauvegardes.

2.1.2 Module réseau multijoueur

1. Choix technologiques : WebRTC + Firebase

J'ai commencé par étudier les différents types de multijoueur possibles en Python : le P2P, le LAN et le online avec serveur dédié. Le modèle P2P a été retenu car il est plus adapté à notre projet : il évite les coûts d'hébergement d'un serveur et reste viable avec seulement deux joueurs.

Je me suis ensuite renseigné sur l'implémentation du P2P en Python. J'ai découvert qu'une connexion entre deux machines sur des réseaux différents pose des problèmes liés au NAT et aux paramètres de sécurité des routeurs. Pour contourner cela, il faut utiliser des protocoles comme STUN, TURN et ICE, regroupés dans la technologie WebRTC. La librairie Python moderne permettant d'utiliser cette technologie est aiortc.

Afin de comprendre son fonctionnement, j'ai développé un mini-jeu de test avec deux joueurs contrôlant chacun une boule dont les déplacements sont synchronisés en temps réel. Le système fonctionne grâce à un échange d'offres et de réponses SDP entre un "host" et un "client", permettant ensuite la création d'un datachannel pour envoyer des messages entre les deux machines.

Cependant, l'échange manuel des clés SDP étant peu pratique, une clé SDP étant une longue suite de caractères difficile à retenir (voir figure 2.1.2.1 de l'annexe). J'ai mis en place une base de données Firebase permettant de stocker les offres et réponses SDP associées à des codes de partie courts, similaires à ceux d'Among Us. Le joueur hôte crée un code unique et y associe son offre SDP, tandis que le client récupère cette offre, génère sa réponse et l'enregistre dans la base de données afin d'établir automatiquement la connexion.

2. Gestion des boucles : threading + asyncio

La boucle de jeu Pygame tourne en continu à cadence fixe et ne peut pas être bloquée par des opérations réseau asynchrones. J'ai donc séparé les deux en deux threads distincts :

- **Thread principal** : boucle de jeu Pygame.
- **Thread secondaire** (daemon) : boucle asyncio dédiée au réseau, avec sa propre instance `asyncio.new_event_loop()`.

La communication entre eux repose sur deux primitives thread-safe de la bibliothèque standard :

- `threading.Event(network_ready)` : indique à la boucle de jeu que la connexion est établie et que la partie peut démarrer.
- `queue.Queue(incoming_messages)` : file dans laquelle le thread réseau dépose les messages reçus. À chaque frame, la boucle de jeu lit l'ensemble des messages présents dans la file et les interprètent les uns après les autres.

3. Protocole de communication hybride

L'envoi des données se fait à une fréquence qu'on peut manuellement adapter dans le code afin d'éviter une trop grande latence et éviter une surcharge réseau. Le protocole de synchronisation distingue deux types de données :

1. **Envoi systématique — données continues** : À chaque cycle réseau, les deux joueurs s'échangent l'état courant de leurs infos propres tels que leur points de vie et leur positions. Le joueur host est le seul à réaliser les calculs pour les ennemis, il est donc le seul à partager les informations des ennemis et le client est le seul à les interpréter.
2. **Système d'actions — événements discrets** : Pour tout ce qui relève d'un événement ponctuel (activer un levier, ouvrir une porte, tirer un projectile, faire une attaque...), un objet de type `Action` est créé localement, exécuté, sérialisé en JSON et envoyé sur le réseau pour être ré-exécuté à l'identique chez l'autre joueur. On envoie l'intention, pas le résultat, ce qui évite de devoir synchroniser tout l'état du monde à chaque changement.

Évolution du modèle d'envoi :

Au départ, le client n'envoyait ses données que lorsqu'il recevait un message du host, ce qui créait des risques de désynchronisation si un paquet était perdu. On a ensuite fait en sorte que les deux envoient indépendamment quoi qu'il arrive, ce qui a résolu les problèmes de synchronisation qu'on avait au début.

4. Robustesse et gestion des déconnexions

- **Arrêt propre** : quand un joueur quitte, un `stop_event` arrête le thread réseau, les tâches async en attente sont annulées, et les deux joueurs retournent au menu.
- **Reset du jeu** : Plusieurs fonctions permettent de reset le jeu afin de pouvoir relancer plusieurs parties sans avoir à complètement relancer le jeu.
- **Gestion des erreurs de code** : si un joueur entre un code invalide, le jeu l'indique et reste sur la page de connexion.
- **Praticité de développement** : afin de faciliter le développement, une simple variable booléenne permettait de désactiver le mode multijoueur pour éviter de devoir lancer plusieurs instances du jeu.

2.1.3. Interface utilisateur et cohérence globale

Afin de rendre le jeu le plus agréable et le plus immersif possible pour l'utilisateur, j'ai développé plusieurs fonctionnalités d'interface utilisateur améliorant la cohérence globale.

1. Menu pause et plein écran

Je me suis chargé de développer le menu pause permettant au joueur d'accéder directement au milieu de la partie aux différentes options et de quitter (voir figure 2.1.3.1 de l'annexe). Pour cela, j'ai réutilisé les différents éléments de menu créés par Antoine Muh notamment les boutons, les titres, les textes et les sliders définis dans `menu_components.py`. Je me suis également chargé de rendre la détection des clics globaux afin d'éviter des potentiels appuis accidentels sur des boutons lors de changements de pages. Ce menu est lui aussi, comme l'ensemble de notre jeu disponible en anglais comme en français. Pour cela, l'ensemble des textes sont enregistrés dans les deux langues dans un dictionnaire lui aussi contenu dans `menu_components.py`.

Les dimensions de l'ensemble des éléments du menu ont également été mis en valeurs relatives afin d'avoir une taille cohérente en fonction de l'écran. Ceci s'est révélé indispensable car j'ai également fait en sorte que le jeu s'affiche en plein écran. Le fait que le jeu occupe l'ensemble de l'écran le rend plus captivant et immersif pour l'utilisateur.

2. HUD

Je me suis également occupé de développer le HUD (Head Up display) permettant au joueur de disposer des informations nécessaires pour évoluer dans le jeu. Une image de celui-ci est disponible dans l'annexe (2.1.3.2)

J'ai notamment créé une barre indiquant ses points de vie avec une couleur dynamique. Au départ je souhaitais faire une barre avec différents cœurs plus ou moins remplis comme dans le jeu Minecraft (voir annexe 2.1.3.3). Mais après discussions avec l'équipe, on a préféré une simple barre du style des barres de chargement.

J'ai également désigné des petits cercles qui servent à informer le joueur sur les délais de réutilisation de ses différentes compétences : ceux-ci se complètent petit à petit et lorsque le cercle est complet, la compétence peut être utilisée. On trouve également au centre de chaque cercle un petit icône indiquant la compétence correspondante.

3. Intégration des animations

J'ai développé une classe `Spritesheet` qui permet de traduire une image représentant une spritesheet contenant les frames d'une animation en une véritable animation visible dans le jeu avec la gestion de la vitesse de l'animation et du changement d'animations selon les actions de l'entité.

`get_animation(row, num_frames, frame_width, frame_height, scale)` retourne une liste de surfaces Pygame pour une animation complète. Le paramètre `scale` effectue le redimensionnement au chargement pour éviter les transformations répétées à chaque frame. Les spritesheets ont été standardisées à 56x56 pixels par frame.

J'ai aussi créé et intégré une animation de sang pour les ennemis afin d'informer les joueurs qu'ils leurs ont infligés des dégâts.

Toutes ces animations ont également dû être synchronisées via le réseau.

4. Autres éléments d'affichage et de cohérence globale

Je me suis également occupé de créer d'autres transitions pour améliorer la fluidité visuelle du jeu notamment en créant une simple animation de fade avec un écran noir afin d'améliorer la transition lors du changement de map ou de la mort des joueurs en complément d'un texte informatif.

C'est également moi qui me suis occupé d'afficher la page de crédit à la fin du jeu après que les joueurs aient fait le choix final avec une direction artistique accordé à ce choix.

2.1.4 Éléments du gameplay et de l'environnement

1. Système de portes

La classe `Door` s'appuie sur la convention de pixels de la carte PNG définie par Guillaume : $R=122$, $40 \leq G \leq 47$, où G encode l'orientation et le sens d'ouverture, et B encode le groupe d'appartenance (lié aux leviers ou autres éléments d'énigmes). J'ai implémenté la lecture de ces pixels lors de la génération de la carte et la création des objets `Door` correspondants. Les portes sont disponibles selon les 4 orientations

afin d'être adaptées dans toutes les situations (voir annexe 2.1.4.1) et sont représentées différemment dans la matrice de la map selon si elles sont ouvertes ou fermées.

Les portes étant associées en groupe en fonction de la salle dans laquelle elles se trouvent, lorsque l'énigme est résolue, la méthode `open_close()` est appliquée sur l'ensemble des portes de la salle afin de libérer l'accès à la suite aux joueurs.

Fonctionnement :

- **Ouverture/fermeture** via `EditMapAction` qui met à jour la matrice de tuiles.
- **Collisions** : une porte fermée est un obstacle, une porte ouverte est franchissable.
- **Réseau** : tout changement d'état est envoyé sur le réseau.

2. Artefacts et changement de map

Je me suis également chargé de créer les artefacts et de les placer sur la map et de faire en sorte de les téléporter eux, ainsi que les joueurs au centre de la carte lorsqu'ils sont récupérés.

Une fois les 4 artefacts récupérés, le changement d'environnement a lieu pour passer sur la carte du combat final avec une transition de fade que j'ai évoquée précédemment.

2.1.5. Tutoriel et assistance à l'utilisateur

1. Tutoriel de l'assignation des touches en check liste

J'ai créé une petite fenêtre de tutoriel au début du jeu qui indique aux joueurs les touches à utiliser pour se déplacer et utiliser les différentes compétences. Cette fenêtre s'affiche de façon fluide avec un fondu d'entrée et un fondu de sortie. Ce tuto est sous la forme d'une check liste et il disparaît une fois que toutes les touches ont été découvertes. Celui-ci est dessiné en pixel art pour suivre la direction artistique globale du jeu. Le fait qu'il soit sous la forme d'une check liste est une façon ludique de faire comprendre au joueur les différentes capacités dont il dispose. Il est lui aussi disponible en français et en anglais.

Une image de ce tuto est disponible dans l'annexe : 2.1.5.1

Il est possible de relancer ce tutoriel à tout moment du jeu via un bouton disponible dans les options du menu pause. Ceci permet à l'utilisateur de se rappeler des touches en milieu de partie s'il les a oubliées.

2. Panneau d'indications pour les énigmes

Nous avons cherché un moyen d'orienter les joueurs vers la résolution d'une énigme sans toutefois leur donner directement la réponse. De plus, il fallait que ceci s'adapte parfaitement dans l'environnement sans paraître complètement inadapté. On a donc décidé de désigner des petits panneaux délabrés que l'on a disposés dans chacune des salles. Les joueurs peuvent interagir avec ces panneaux et ainsi voir en plein écran ce qu'il y a écrit dessus. Sur chaque panneau on a donc écrit un petit texte indiquant ce que les joueurs doivent faire pour résoudre l'énigme sans donner explicitement la solution.

Ces panneaux ont été désignés comme délabrés et avec une police d'écriture dans le style dark fantasy afin de bien s'intégrer dans l'environnement global du jeu et ne pas paraître inadapté.

Plusieurs images des panneaux sont visible dans l'annexe : 2.1.5.2

2.1.6 Génération de l'exécutable et sauvegarde

1. Génération de l'exécutable

Afin d'éviter tout problème avant la soutenance, je m'y suis pris un peu à l'avance pour découvrir comment générer l'exécutable finale pour notre jeu. Tout d'abord, j'ai dû faire un refactoring complet de l'architecture du projet. En effet les fichiers de code pythons se trouvaient tout à la racine et il devenait compliqué de s'y retrouver avec le nombre grandissant de fichiers. De plus, pour des raisons de dépendances, il fallait organiser les différents fichiers en packages. La structure finale du projet est visible dans l'annexe 2.1.6.1

Le refactoring en packages a été fait pour permettre la génération de l'exécutable via PyInstaller. Avec tous les fichiers `.py` à la racine, la configuration du `.spec` de PyInstaller ne pouvait pas résoudre correctement les dépendances. Ce fichier `.spec` permet de définir les dépendances et de dire à PyInstaller comment compiler le projet. PyInstaller est la librairie qui permet de générer l'exécutable.

De plus, les chemins menant aux différents assets n'étant pas les mêmes lorsque le projet est sous la forme de code source et sous la forme d'exécutable, j'ai donc développé la fonction `resource_path()` dans `config/settings.py` qui gère la différence de chemins entre le mode développement et l'exécutable PyInstaller.

De plus, pour faciliter l'intégration et l'installation des différentes dépendances du projet, j'ai regroupé toutes les librairies utilisées dans le fichier `requirements.txt`.

2. Enregistrement et chargement de sauvegardes

J'ai développé en collaboration avec Guillaume l'enregistrement et le chargement de sauvegarde dans notre jeu. Il s'est chargé de récupérer toutes les informations du jeu pour les transformer en un grand dictionnaire python et aussi de charger un dictionnaire de ce type pour répercuter les données sur le jeu. Moi je me suis chargé de transformer ce dictionnaire en fichier JSON et de l'enregistrer au bon endroit.

Au départ je souhaitais enregistrer les sauvegardes dans le dossier `%APPDATA%` comme toutes les autres applications. Mais j'ai eu des problèmes de droits d'écriture à cet endroit donc j'ai préféré écrire le dossier de sauvegardes directement dans le dossier Document de l'utilisateur. De plus, ça donne plus facilement accès aux fichiers JSON pour que les joueurs débrouillards puissent modifier leur avancée à la main.

Le nom de fichier est sanitisé (alphanumériques, espaces et tirets uniquement).

En collaboration avec Antoine Muh j'ai également intégré le chargement des saves les plus récentes via des boutons.

On sauvegarde les positions et points de vie de toutes les entités, mais aussi l'état complet des énigmes : état des leviers, des portes, et des puzzles. Ça permet une vraie reprise de partie et pas juste un retour à la dernière zone.

Lors du chargement d'une sauvegarde, l'état est propagé au joueur distant via le réseau pour que les deux repartent dans le même état via un message de départ : la save est entièrement envoyée via le réseau pour être sûr que les deux joueurs sont synchronisés.

2.1.7 Conclusion

Ce projet m'a permis d'apprendre de nombreuses choses et d'acquérir de nouvelles compétences à la fois purement techniques au niveau du code, plus spécifiquement dans l'aspect réseau, mais aussi dans la gestion d'un projet et le travail en équipe.

En effet, j'avais déjà créé des jeux en python auparavant mais pas avec pygame et pas en multijoueur. J'ai dû apprendre à m'adapter à cette nouvelle librairie pour rechercher des ressources pour mener à bien nos objectifs. L'aspect réseau du jeu apporte de nombreuses contraintes auxquelles on ne pense pas dès le départ, notamment pour la synchronisation des différents éléments et de la potentielle latence. Mais la dimensions multijoueur rend le jeu bien plus intéressant et agréable à jouer. Dans le cas de notre jeu, c'était tout simplement indispensable.

La collaboration au niveau du code était aussi nouvelle pour moi, c'est le premier projet de cette ampleur ou j'ai dû travailler et coder avec d'autres personnes sur le même projet. On a fait face à de nombreux conflits de code mais on a réussi à les surmonter et à produire au final un jeu qui selon moi est toujours améliorable mais est plus que satisfaisant : on a respecté nos objectifs, et malgré quelques bug qu'on a sûrement pas encore découvert, on a un beau jeu fluide qui respecte les attentes.

2.1.8 Ressources

Voici les différentes ressources que j'ai utilisé pour mener à bien ce projet :

- <https://code.tutsplus.com/building-a-peer-to-peer-multiplayer-networked-game--gamedev-10074t>
- <https://aiortc.readthedocs.io/en/latest/>
- <https://dev.to/whitphx/python-webrtc-basics-with-aiortc-48id>
- <https://www.pygame.org/docs/>
- https://www.youtube.com/watch?v=M6e3_8LHc7A
- <https://www.piskelapp.com/>

2.2. Guillaume KLEIN :

2.2.1 Introduction

Cette partie approfondit les contributions techniques réalisées sur le moteur du jeu, les systèmes de gameplay ainsi que l'architecture logicielle générale du projet. L'objectif principal du développement était de construire un moteur coopératif isométrique entièrement modulaire, capable de gérer simultanément le rendu, les interactions environnementales, les énigmes, les collisions, les déplacements, les compétences et la synchronisation multijoueur.

Le projet repose sur une idée centrale : séparer autant que possible la logique du jeu, l'affichage, les interactions et l'état global du monde. Cette séparation permet d'éviter qu'un système soit dépendant directement d'un autre. Par exemple, un joueur ne modifie pas directement une porte ou un levier ; il crée une action, cette action est ajoutée dans une file d'exécution, puis le contexte de jeu l'exécute au bon moment. Ce fonctionnement rend le code plus stable, plus lisible et plus facile à synchroniser en multijoueur.

2.2.2 Le moteur isométrique

Le moteur repose principalement sur trois couches complémentaires : une représentation logique du monde sous forme de matrice 3D, un système de conversion géométrique entre coordonnées cartésiennes et coordonnées isométriques, puis une couche d'actions orientée objet permettant de déclencher les conséquences des interactions.

La matrice du monde stocke l'intégralité des informations utiles au gameplay. Chaque case de la carte contient plusieurs niveaux de hauteur, représentés par l'indice `z`. Une case de la matrice est donc manipulée sous la forme `map[x][y][z]`. Cette structure permet de représenter un sol, un objet posé sur ce sol et éventuellement un élément en hauteur, tout en conservant une grille logique simple. Le rendu graphique se charge ensuite d'interpréter ces identifiants numériques pour afficher les bons sprites.

Le moteur de rendu applique ensuite les transformations géométriques nécessaires afin de convertir ces coordonnées logiques en coordonnées visibles à l'écran. La conversion cartésien vers isométrique utilisée dans `cart_to_iso()` est la suivante :

```
screen_x = (x - y) * (TILE_WIDTH // 2) + 400
screen_y = (x + y) * (TILE_HEIGHT // 2) - z * TILE_HEIGHT
```

Dans cette formule, `x` et `y` représentent la position de la tuile dans la matrice, tandis que `z` représente le niveau de hauteur. Le terme `(x - y)` produit le décalage horizontal typique d'une vue isométrique : deux tuiles situées sur des diagonales opposées s'écartent visuellement à gauche ou à droite. Le terme `(x + y)` produit quant à lui la descente verticale progressive de la carte, ce qui donne l'impression de profondeur. Enfin, le terme `- z * TILE_HEIGHT` fait remonter l'élément affiché à l'écran lorsqu'il se trouve sur une couche supérieure. Cela permet notamment d'afficher correctement les murs ou certains objets hauts.

L'offset `+ 400` appliqué à `screen_x` sert à recentrer la carte à l'écran. Sans cet offset, la projection isométrique pourrait être partiellement hors champ selon la position des tuiles.

Le moteur utilise également la conversion inverse avec `iso_to_cart_tile()` afin de retrouver la case logique correspondant à une position isométrique. Cette conversion est indispensable pour les déplacements, les collisions, les interactions et les projectiles. Le calcul inverse est :

```
iso_x = screen_x - 400
iso_y = screen_y + z * TILE_HEIGHT
cart_x = (iso_y / (TILE_HEIGHT / 2) + iso_x / (TILE_WIDTH / 2)) / 2
cart_y = (iso_y / (TILE_HEIGHT / 2) - iso_x / (TILE_WIDTH / 2)) / 2
```

Cette inversion permet de passer d'une position affichée à l'écran vers une position dans la matrice. Dans certains cas, la fonction renvoie directement des entiers pour accéder à la case de la carte. Dans d'autres cas, elle peut renvoyer des valeurs décimales grâce au paramètre `decimals=True`. Cette précision décimale est très importante pour les collisions fines, car elle permet de savoir où se trouve exactement le joueur à l'intérieur d'une tuile.

2.2.3 Développement avancé du système de sauvegarde

Le système de sauvegarde développé autour du `GameStateRegistry` constitue un élément essentiel du projet. L'objectif n'était pas seulement de sauvegarder quelques variables, mais de pouvoir reconstruire l'état exact du monde après le rechargement d'une partie. Pour cela, le registre centralise les éléments persistants du jeu et expose deux méthodes principales : `to_dict()` pour exporter l'état du monde, et `load_dict()` pour le restaurer.

La méthode `to_dict()` transforme l'état du jeu en dictionnaire Python sérialisable en JSON. Cette approche est adaptée car un fichier JSON permet de stocker des types simples : nombres, chaînes de caractères, booléens, listes et dictionnaires. Plutôt que d'essayer d'enregistrer directement des instances Python complexes, chaque objet important du monde est résumé sous forme de données minimales permettant de le reconstruire.

Pour les leviers, le registre sauvegarde l'identifiant du groupe, l'identifiant du levier, son état (``state``) et son verrouillage (``locked``). Pour les tuiles du puzzle inspiré du Jeu de la Vie, le même principe est appliqué : chaque tuile conserve son état et son verrouillage. Pour les portes, le système sauvegarde leur état d'ouverture ou de fermeture. Pour les énigmes laser, le registre sauvegarde surtout l'orientation des miroirs, car cette orientation suffit ensuite à recalculer tout le trajet des faisceaux lumineux. Les artefacts sont sauvegardés par un booléen ``obtained``, indiquant s'ils ont déjà été récupérés ou non. Enfin, les `global_flags` permettent de conserver des événements plus généraux de progression.

Le chargement avec `load_dict()` est plus complexe qu'une simple réaffectation de valeurs. En effet, il ne suffit pas de remettre les données logiques dans les objets : il faut aussi mettre à jour l'affichage dans la matrice de la carte. Par exemple, lorsqu'un levier est restauré, sa méthode `update_matrix(game_context.map)` est appelée afin que le sprite affiché corresponde à son état réel. Le même principe est appliqué aux tuiles du puzzle avec `update_tile_visual(game_context.map)`.

Le cas des portes est intéressant car le code ne force pas simplement une valeur dans la matrice. Il vérifie si l'état actuel de la porte est différent de l'état sauvegardé. Si c'est le cas, il exécute l'action `open_close(game_context)`. Cette approche permet de conserver toute la logique déjà présente dans l'ouverture et la fermeture des portes, au lieu de dupliquer le comportement dans le système de sauvegarde.

Pour les lasers, le chargement restaure d'abord l'orientation de chaque miroir :

```
mirror.orientation = tuple(mirror_data["orientation"])
mirror.update_visual(game_context.map)
```

Ensuite, le registre appelle `refresh_all_lasers(game_context.map)`. Cette fonction supprime les anciennes traces lumineuses dans la matrice, réinitialise les récepteurs, puis recalcule entièrement les faisceaux à partir des sources. Concrètement, les tuiles représentant les lasers (``95``, ``96``, ``97``) sont remises à zéro, les récepteurs activés (``94``) reviennent à leur état normal (``93``), puis chaque source recalcule son rayon. Ce recalcul complet garantit que l'état visuel final est cohérent avec l'orientation des miroirs restaurée depuis la sauvegarde.

Le système de sauvegarde montre donc une idée importante : le registre ne se contente pas de stocker des valeurs. Il sert aussi de point de reconstruction du monde, en assurant la cohérence entre les données sauvegardées, la logique des objets et leur représentation visuelle.

2.2.4 Architecture des interactions et du gameplay

2.2.4.1 Centralisation des interactions via `InteractAction`

J'ai développé une architecture centralisée permettant de gérer les interactions du joueur avec l'environnement à travers une action unique : `InteractAction`. Le but est que le joueur n'ait pas à savoir directement s'il interagit avec un levier, une tuile, un miroir, un panneau ou un artefact. Il déclenche simplement une interaction, puis le système détermine automatiquement quel objet est concerné.

Lorsqu'une interaction est exécutée, la position isométrique du joueur est convertie en coordonnées cartésiennes grâce à `iso_to_cart_tile()`. Cette étape est indispensable, car les objets interactifs sont stockés dans la matrice avec leurs coordonnées logiques (`x`, `y`). Si l'action vient du réseau, les coordonnées reçues sont déjà cartésiennes et n'ont pas besoin d'être reconverties.

Ensuite, plusieurs méthodes de vérification sont appelées : `leverVerif()`, `tilePuzzleVerif()`, `artefactVerif()`, `signVerif()` et `mirrorVerif()`. Chacune parcourt une partie du `GameStateRegistry` pour voir si un objet interactif se trouve à proximité de la position du joueur.

Le système des artefacts utilise une zone d'interaction plus large qu'une simple case. Le code vérifie les neuf cases autour du joueur.

Cette méthode augmente la portée d'interaction avec les artefacts et évite qu'un joueur doive être placé exactement sur une seule case précise pour les récupérer.

Pour les miroirs des énigmes laser, la vérification est également faite dans une zone autour du joueur. Le système vérifie si le miroir est proche et s'il appartient au bon personnage. En effet, certains miroirs sont associés à Aeden, d'autres à Lyra. Cette contrainte renforce l'aspect coopératif du jeu car les deux joueurs n'ont pas les mêmes possibilités d'action.

Une fois l'objet détecté, l'action appropriée est ajoutée au contexte de jeu : `LeverAction` pour un levier, `tilePuzzleAction` pour une tuile du puzzle, `ArtefactAction` pour un artefact, ou une rotation directe pour un miroir. Cette architecture permet de conserver une touche d'interaction unique côté joueur, tout en gardant un système extensible côté moteur.

2.2.4.2 Système de compétences du joueur

Le système de compétences du joueur a été conçu pour éviter de coder directement chaque attaque ou interaction dans la boucle principale du jeu. L'objectif était de créer une architecture modulaire, où chaque compétence possède son propre comportement, tout en partageant une base commune. Pour cela, toutes les compétences héritent d'une classe mère appelée `Skill`.

Cette classe mère centralise les éléments communs à toutes les compétences.

Chaque compétence possède donc trois informations principales. Le `cooldown` représente le temps de recharge total de la compétence. `current_cd` représente le temps restant avant que la compétence puisse être utilisée. Enfin, `range` représente la portée de la compétence, c'est-à-dire la distance à laquelle elle peut produire un effet.

Cette structure permet de gérer de la même manière une attaque à l'épée, une attaque à distance, une interaction ou une compétence spéciale. Même si les effets sont très différents, leur logique d'activation reste commune.

À chaque frame, la méthode `update()` est appelée pour chaque compétence du joueur, elle retire 1 à `self.current_cd` afin de réduire le cooldown jusqu'à zéro.

Cela signifie que le cooldown est géré comme un compteur de frames. Tant que `current_cd` est supérieur à zéro, il diminue progressivement. Quand il atteint zéro, la compétence redevient disponible. Ce choix est simple et efficace dans un jeu Pygame, car la boucle principale tourne régulièrement à un rythme fixe.

Le joueur possède une liste de compétences : `self.skills = [SwordAttack(), Interact(self.host), LongAttack(self)]`

Cette liste permet d'utiliser les compétences par index. Dans la boucle principale, lorsqu'une touche est pressée, le joueur appelle :

```
action = playerL.try_use(index)
```

Par exemple, l'index 0 correspond à l'attaque au corps à corps, l'index 1 à l'interaction, et l'index 2 à l'attaque à distance. Si le joueur est Aeden, une quatrième compétence est ajoutée :

```
if self.host: self.skills.append(LightAura())
```

Cela permet d'avoir des compétences communes aux deux joueurs, tout en ajoutant des compétences spécifiques à un personnage.

La méthode `try_use()` de la classe `Player` sert d'intermédiaire entre les inputs et les compétences. Elle appelle la méthode `try_use()` de la compétence demandée :

```
return self.skills[index].try_use(self)
```

La compétence reçoit alors le joueur qui l'utilise, appelé `caster`. Ce paramètre est important car l'action créée doit connaître la position du joueur, sa direction, ou encore son identité.

La méthode `try_use()` de `Skill` vérifie d'abord si la compétence est disponible :

```
def can_use(self):
    return self.current_cd <= 0
```

Si le cooldown n'est pas terminé, la compétence ne produit rien :

```
if not self.can_use():
    return None
```

Si la compétence est disponible, le cooldown est réinitialisé : `self.current_cd = self.cooldown`

Puis la compétence crée son action : `return self.create_action(caster)`

La méthode `create_action()` n'est pas définie directement dans `Skill`. Elle sert de méthode abstraite :

```
def create_action(self, caster):
    raise NotImplementedError("create_action must be overridden")
```

Cela signifie que chaque compétence concrète doit définir son propre comportement. La classe mère impose une structure commune, mais laisse les classes enfants décider de l'effet exact.

Par exemple, la compétence `SwordAttack` définit une attaque au corps à corps :

```
class SwordAttack(Skill):
    def __init__(self):
        super().__init__(cooldown=50, range=1)
    def create_action(self, caster):
        caster.anim_status = "close range attack"
        return MeleeAction(caster, self.range)
```

Lorsqu'elle est utilisée, elle change d'abord l'état d'animation du joueur en "close range attack". Cela permet de jouer la spritesheet d'attaque correspondante. Ensuite, elle crée une `MeleeAction`, qui se chargera réellement de détecter les ennemis touchés et d'appliquer les dégâts. Cette séparation est importante : la compétence ne gère pas directement les collisions. Elle dit seulement : "le joueur veut faire une attaque au corps à corps". L'action correspondante s'occupe ensuite de l'exécution concrète.

La compétence d'interaction fonctionne de manière similaire. Elle possède un `cooldown` de 40, ce qui évite que le joueur puisse déclencher trop rapidement plusieurs interactions successives. Lorsqu'elle est utilisée, elle lance l'animation d'interaction puis crée une `interactAction`. Cette action recevra la position du joueur et l'identité du personnage afin de déterminer quel objet peut être activé. L'attaque à distance est gérée par `LongAttack`. Cette compétence possède un `cooldown` plus élevé, car elle permet d'attaquer à distance. Lors de son utilisation, elle lance l'animation "long range attack" et crée une `ProjectileAction`. Le booléen envoyé à `ProjectileAction` permet de savoir si le projectile appartient à Aeden ou à Lyra, ce qui permet ensuite d'utiliser la bonne image de projectile. Enfin, la compétence `LightAura` fonctionne un peu différemment. Elle hérite aussi de `Skill`, mais elle ne crée pas une action classique. Elle agit directement sur l'état du joueur :

```
caster.light_aura = not caster.light_aura
```

Cette compétence fonctionne comme une bascule. Lorsqu'Aeden l'utilise, son état `light_aura` passe de faux à vrai, ou de vrai à faux. Cette différence montre que le système reste flexible : la plupart des compétences créent des actions, mais certaines peuvent aussi modifier directement un état interne lorsqu'il s'agit d'un effet spécial.

Dans la boucle principale, les compétences sont liées aux touches du clavier. Par exemple, la barre espace déclenche l'attaque au corps à corps :

```
if keys[pygame.K_SPACE]:
    action = playerL.try_use(0)
    if action: context.add_action(action)
```

La touche E déclenche l'interaction :

```
if keys[pygame.K_e]:
    action = playerL.try_use(1)
    if action: context.add_action(action)
```

Et la touche X déclenche l'attaque à distance :

```
if keys[pygame.K_x]:
    action = playerL.try_use(2)
    if action: context.add_action(action)
```

Le point important est que la boucle principale ne sait pas comment fonctionne chaque compétence. Elle demande simplement au joueur d'essayer d'utiliser une compétence. Si une action est renvoyée, elle est ajoutée dans la file d'actions du contexte.

Cette file d'actions est ensuite exécutée plus tard par le jeu. Cela permet de garder une organisation claire : les inputs déclenchent des compétences, les compétences créent des actions, et les actions modifient le monde.

Le système de `cooldown` est également utilisé dans l'interface du joueur. Dans le HUD, les compétences sont affichées avec des cercles de recharge. Le pourcentage d'avancement est calculé avec :

```
(current_cd / cooldown)
```

Quand `current_cd` est proche de `cooldown`, la compétence vient d'être utilisée et le cercle est presque vide. Quand `current_cd` approche de zéro, le cercle se remplit progressivement. Cela donne au joueur un retour visuel clair sur la disponibilité de ses compétences.

Cette architecture présente plusieurs avantages. D'abord, elle rend les compétences faciles à équilibrer : il suffit de modifier les valeurs de cooldown ou de portée. Ensuite, elle rend le code extensible : pour ajouter une nouvelle compétence, il suffit de créer une nouvelle classe héritant de `Skill`. Enfin, elle reste compatible avec le système réseau, car les compétences produisent des actions qui peuvent ensuite être envoyées à l'autre joueur.

Le système de compétences sert donc de pont entre les contrôles du joueur et la logique globale du jeu. Il transforme une entrée clavier en intention de gameplay, puis en action exécutable par le moteur.

2.2.4.3 Aura lumineuse et coopération asymétrique

La compétence `LightAura` a été ajoutée comme une capacité spécifique à Aeden. Elle ne correspond pas à une attaque classique, mais plutôt à une mécanique de soutien pensée pour renforcer la coopération entre les deux joueurs. L'objectif est de donner aux deux personnages des rôles différents : Lyra peut traverser certaines zones particulières, tandis qu'Aeden peut l'aider indirectement grâce à la lumière.

Dans la classe `Player`, les deux personnages possèdent une liste de compétences commune :

```
self.skills = [SwordAttack(), Interact(self.host), LongAttack(self)]
```

Cette liste contient les compétences de base : l'attaque au corps à corps, l'interaction avec l'environnement et l'attaque à distance. Cependant, lorsqu'un joueur est identifié comme `host`, c'est-à-dire Aeden dans notre logique, une compétence supplémentaire est ajoutée :

```
if self.host: self.skills.append(LightAura())
```

Cette condition permet de rendre la compétence exclusive à Aeden. Lyra ne possède donc pas cette capacité dans sa liste de compétences. Ce choix est important car il évite d'avoir deux personnages identiques et permet de construire des énigmes dans lesquelles chacun possède une fonction précise.

La compétence `LightAura` hérite de la classe mère `Skill`, comme les autres compétences du joueur. Elle possède donc un cooldown, mais contrairement aux attaques, elle n'a pas besoin de portée réelle. Sa portée est donc définie à 0 : `super().__init__(cooldown=100, range=0)`

La classe contient également une durée : `self.duration = duration` et `self.timer = 0` et `self.active = False`

Ces attributs servent à représenter l'état de l'aura. `duration` correspond à la durée prévue de l'effet, `timer` peut servir à compter le temps restant, et `active` indique si la compétence est actuellement en cours d'utilisation.

Lorsqu'Aeden utilise cette compétence, la méthode `try_use()` vérifie d'abord si le cooldown est terminé

```
if not self.can_use():  
    return False
```

Si la compétence est disponible, le cooldown est réinitialisé :

```
self.current_cd = self.cooldown
```

Puis l'aura est activée : `self.active = True` et `self.timer = self.duration` et `caster.light_aura = not caster.light_aura`

Le point important est la dernière ligne : au lieu d'appliquer simplement un effet ponctuel, la compétence inverse l'état `light_aura` du joueur. Si l'aura était désactivée, elle devient active ; si elle était déjà active, elle se désactive. Cela donne à la compétence un comportement de bascule, ce qui permet à Aeden de choisir quand entrer ou sortir de cet état.

Lorsque `light_aura` est active, le déplacement du joueur est volontairement bloqué dans la méthode

```
detect_movement():  
if self.light_aura:  
    self.direction = (0, 0)
```

```
self.is_moving = False
return
```

Ce passage du code est important pour l'équilibrage. Aeden ne peut pas à la fois se déplacer librement et produire son aura lumineuse. Il devient temporairement un point fixe de soutien. La direction est remise à (0, 0), l'état de mouvement passe à False, puis la fonction s'arrête immédiatement avec `return`.

Cela transforme la compétence en vraie décision de gameplay. Aeden doit choisir entre continuer à se déplacer ou s'arrêter pour aider Lyra. L'aura lumineuse n'est donc pas seulement un effet visuel : elle impose une contrainte au joueur qui l'utilise.

Dans la boucle principale du jeu, cet état est ensuite utilisé pour modifier l'affichage et le comportement de la caméra. Lorsque `playerL.light_aura` est actif, la caméra ne suit plus Aeden, mais se centre sur Lyra :

```
if playerL.light_aura:
    context.camera.follow(x_player2, y_player2)
else:
    context.camera.follow(x_player1, y_player1)
```

Ce choix est directement lié à la mécanique du labyrinthe. Lorsque Lyra se trouve dans une zone sombre, Aeden peut activer son aura pour que l'affichage se concentre sur elle. Cela permet au joueur qui contrôle Aeden de devenir un soutien visuel pour Lyra, même s'il ne se déplace plus lui-même.

L'aura est ensuite affichée dans `update_game()` avec :

```
if playerL.host:
    if playerL.light_aura:
        draw_light_aura(context)
```

La fonction `draw_light_aura()` crée une surface transparente sur tout l'écran, puis dessine progressivement des rectangles blancs de plus en plus petits depuis les bords vers le centre. Cela produit un effet de halo lumineux autour de l'écran :

```
for i in range(100):
    alpha = int(max_alpha * (1 - i / 100))
    pygame.draw.rect(aura_surface, (255, 255, 255, alpha), (i, i, width - 2*i,
height - 2*i), 2)
```

Le calcul de l'alpha permet d'obtenir un dégradé. Au début de la boucle, `i` est faible, donc l'alpha est proche de `max_alpha`. Plus `i` augmente, plus l'alpha diminue. Visuellement, cela crée une aura lumineuse progressive, et non un simple cadre blanc fixe.

Cette compétence est fortement liée aux zones d'obscurité. Dans le système de carte, certaines tuiles possèdent l'identifiant 101, ce qui active l'état darkness de Lyra lorsqu'elle se trouve dessus. Lorsque Lyra est dans l'obscurité, son écran est assombri par `draw_darkness()`. Aeden peut alors utiliser son aura lumineuse pour l'aider à se repérer.

Le fonctionnement repose donc sur une complémentarité claire :

Aeden possède l'aura lumineuse, mais doit s'arrêter pour l'utiliser. Lyra peut traverser certaines zones spéciales et explorer le labyrinthe, mais elle peut être gênée par l'obscurité. La progression dépend donc de la communication entre les deux joueurs.

Cette mécanique crée une coopération asymétrique : les deux joueurs ne font pas la même chose au même moment, mais leurs actions sont complémentaires. Aeden joue un rôle de guide ou de soutien, tandis que Lyra prend le risque d'avancer dans les zones sombres.

D'un point de vue technique, ce système est intéressant car il réutilise plusieurs briques déjà existantes du moteur :

- la classe `Skill` pour gérer le cooldown ;
- l'état `light_aura` dans `Player` ;
- la boucle principale `update_game()` pour adapter la caméra ;
- les fonctions d'affichage pour produire l'effet visuel ;
- la matrice de la carte pour déclencher les zones d'obscurité.

L'aura lumineuse est donc un bon exemple de mécanique construite par combinaison de systèmes simples. Elle ne repose pas sur un bloc isolé de code, mais sur l'interaction entre le joueur, les compétences, le rendu, la caméra et la carte.

2.2.5 Gestion avancée des déplacements et collisions

2.2.5.1 Gestion des pieds du joueur

Dans un moteur isométrique, il est rarement pertinent d'utiliser tout le rectangle du sprite comme zone de collision. Les sprites sont visuellement grands, mais seule une petite partie correspond réellement au contact avec le sol. J'ai donc choisi de baser les collisions du joueur sur la position de ses pieds.

La méthode `deduce_feet_from_iso_coords()` renvoie deux points à partir de la position isométrique du joueur :

```
left_foot = (player_x - 32, player_y)
right_foot = (player_x + 1, player_y)
```

Le décalage de `-32` correspond à l'écart approximatif entre les deux pieds sur le sprite. Le pied droit est représenté par la position principale du joueur avec un léger décalage `+1`. Ces deux points servent ensuite de références pour tester les collisions.

Lorsqu'un déplacement est demandé, la méthode `detect_movement()` calcule d'abord un vecteur (dx, dy) à partir des touches ZQSD. Si aucune touche n'est pressée, le joueur est considéré comme immobile. Si une direction est pressée, le système calcule ensuite un vecteur réel adapté à la vue isométrique.

Le moteur corrige certaines diagonales :

- bas-droite : $(dx, dy) = (1, 1)$ devient $(1, 0.5)$
- bas-gauche : $(dx, dy) = (-1, 1)$ devient $(-1, 0.5)$
- haut-droite : $(dx, dy) = (1, -1)$ devient $(1, -0.5)$
- haut-gauche : $(dx, dy) = (-1, -1)$ devient $(-1, -0.5)$

Cette correction est nécessaire car, à l'écran, les lignes de la grille isométrique n'ont pas la même pente que dans une grille cartésienne classique. Sans cette correction, le joueur ne suivrait pas correctement l'orientation visuelle des tuiles.

Si la touche Shift est enfoncée, le déplacement réel est multiplié par `1.5`. Le sprint est donc simplement un facteur appliqué au vecteur de mouvement :

```
real_dx = real_dx * 1.5  
real_dy = real_dy * 1.5
```

Le moteur calcule ensuite les futures positions des deux pieds :

```
left_future = left_foot + (real_dx * speed, real_dy * speed)  
right_future = right_foot + (real_dx * speed, real_dy * speed)
```

Le joueur ne bouge que si `foot_can_move()` renvoie vrai pour les deux pieds. Cette méthode convertit la position isométrique du pied en coordonnées de matrice avec `iso_to_cart_tile(x, y, decimals=True)`, puis appelle `is_walkable()`.

Cette logique empêche qu'un seul pied traverse un mur pendant que l'autre reste dans une zone valide. Elle rend donc les collisions bien plus précises qu'un système basé sur un seul point central.

2.2.5.2 Collisions partielles sur les portes

Certaines portes du jeu n'occupent qu'une partie de leur tuile. Une tuile classique est soit traversable, soit bloquante, mais ce modèle est insuffisant pour les portes isométriques qui peuvent être placées sur une diagonale de la case.

Pour résoudre ce problème, j'utilise les coordonnées décimales retournées par `iso_to_cart_tile(..., decimals=True)`. La partie entière indique la case de la matrice, tandis que la partie décimale indique la position relative du pied à l'intérieur de cette case.

Dans `is_walkable()`, les portes utilisent des identifiants de tiles situés entre 20 et 27. Le code teste alors la valeur décimale de `x_float` ou `y_float` selon l'orientation de la porte. Par exemple :

```
si tile_2 == 24 et y_float % 1 >= 0.1 alors la zone est traversable  
si tile_2 == 25 et y_float % 1 <= 0.9 alors la zone est traversable  
si tile_2 == 26 et x_float % 1 >= 0.1 alors la zone est traversable  
si tile_2 == 27 et x_float % 1 <= 0.9 alors la zone est traversable
```

Le modulo `% 1` permet de récupérer uniquement la partie décimale. Une valeur proche de 0 signifie que le joueur est proche d'un bord de la tuile, tandis qu'une valeur proche de 1 signifie qu'il est proche de l'autre bord. En fixant des seuils comme 0.1 ou 0.9, le moteur définit quelle partie de la tuile est bloquante et quelle partie est traversable.

Cette méthode permet de conserver une matrice simple tout en obtenant une collision plus fine. Le moteur n'a pas besoin de subdiviser chaque tuile en sous-cases : il utilise directement la position relative du joueur dans la case.

2.2.5.3 Zones spéciales : Lyra et obscurité

Le système de déplacement ne se limite pas à vérifier si une tuile est simplement traversable ou bloquante. Il intègre aussi des zones spéciales liées directement aux mécaniques coopératives du jeu, notamment les zones réservées à Lyra et les zones d'obscurité.

Ces zones sont définies dès la génération de la carte à partir du pixel art. Dans `image_to_matrix()`, certaines couleurs précises sont reconnues par des fonctions dédiées. Par exemple, une zone réservée à Lyra est détectée par `is_lyra_only(pixel)`, puis traduite dans la matrice avec l'identifiant 100 :

```
matrix[x][y][1] = 100
matrix[x][y][2] = 100
matrix[x][y][0] = 150
```

Cette case devient alors une tuile spéciale, représentée par de la fumée mauve. Elle n'est pas seulement décorative : son identifiant est ensuite utilisé dans la logique de collision du joueur.

Dans la méthode `is_walkable()` de la classe `Player`, le code vérifie si la tuile correspond à une zone Lyra-only :

```
elif self.host == False and 100 == tile:
    return True
```

Ici, `self.host == False` correspond au joueur Lyra. Cela signifie que si la tuile vaut 100, seul le joueur qui n'est pas l'hôte peut la traverser. Aeden, lui, ne valide pas cette condition et la tuile reste donc bloquante pour lui.

Ce système permet de créer des chemins réservés à un personnage sans ajouter une logique complexe dans la carte. Le level design peut simplement placer une couleur précise dans l'image pixel art, et le moteur transforme automatiquement cette zone en passage réservé à Lyra.

Cette mécanique est importante dans le cadre d'un jeu coopératif asymétrique. Les deux joueurs ne possèdent pas exactement les mêmes possibilités de déplacement. Lyra peut accéder à certaines zones interdites à Aeden, ce qui oblige les joueurs à se répartir les rôles et à coopérer pour progresser.

Le système d'obscurité fonctionne selon le même principe. Dans la génération de carte, la fonction `is_darkness_lyra(pixel)` détecte les pixels correspondant aux zones sombres. Ces cases sont ensuite placées dans la matrice avec l'identifiant 101 sur la couche basse :

```
matrix[x][y][1] = 0
matrix[x][y][2] = 0
matrix[x][y][0] = 101
```

Contrairement à une porte ou à un mur, cette tuile n'est pas forcément bloquante. Elle sert plutôt de marqueur logique pour assombrir la vision du personnage. Le moteur utilise cette valeur pour savoir si le joueur se trouve dans une zone d'obscurité.

La méthode `detect_darkness()` convertit la position isométrique du joueur en coordonnées de matrice :

```
p_x, p_y = iso_to_cart_tile(self.x, self.y)
```

Elle vérifie ensuite la couche basse de la tuile :

```
if tiles[p_x][p_y][0] == 101:
    self.darkness = True
else:
    self.darkness = False
```

La valeur 101 agit donc comme un indicateur de zone sombre. Si le joueur est sur cette tuile, son attribut `darkness` devient vrai. Sinon, il redevient faux.

Cette information est ensuite utilisée dans la boucle principale du jeu. Dans `update_game()`, si le joueur local possède `darkness == True`, la fonction `draw_darkness(context)` est appelée. Cette fonction ajoute un voile noir sur l'écran, tout en laissant une zone visible autour du centre :

```
dark_surface.fill((0, 0, 0, 255))
radius = 120 + random.randint(-8, 8)
```

Le rayon varie légèrement avec `random.randint(-8, 8)`, ce qui crée un effet de tremblement visuel. L'obscurité n'est donc pas parfaitement statique : elle donne une impression plus organique et renforce l'ambiance du labyrinthe.

Le principe est que la matrice ne sert pas uniquement à afficher des tuiles. Elle contient aussi des informations de gameplay invisibles ou semi-visibles. Une case peut donc être utilisée comme marqueur logique pour déclencher un effet de jeu.

Ce système permet de créer une vraie complémentarité entre les personnages. Lyra peut traverser certaines zones et entrer dans l'obscurité, tandis qu'Aeden peut utiliser son aura lumineuse pour l'aider. L'obscurité devient alors une mécanique coopérative, et pas seulement un effet visuel.

L'avantage principal de cette architecture est sa simplicité côté level design. Pour créer une zone spéciale, il suffit de placer la bonne couleur dans l'image pixel art. Le moteur se charge ensuite de convertir cette couleur en identifiant de tuile, puis la classe `Player` et la boucle principale interprètent cet identifiant pour appliquer les règles de gameplay correspondantes.

2.2.6 Développement du système de projectiles

2.2.6.1 Projectiles des joueurs

Le système de projectiles des joueurs a été conçu comme une extension directe du système de compétences et d'actions. L'objectif était d'ajouter une attaque à distance sans casser l'architecture déjà mise en place pour les autres compétences. Pour cela, le projectile n'est pas créé directement dans la boucle principale du jeu ni directement par la classe `Player`. Il passe par plusieurs étapes : une compétence, une action, puis un objet projectile physique.

Lorsqu'un joueur appuie sur la touche associée à l'attaque à distance, la boucle principale appelle :

```
action = playerL.try_use(2)
```

L'index 2 correspond à la compétence `LongAttack` dans la liste des compétences du joueur :

```
self.skills = [SwordAttack(), Interact(self.host), LongAttack(self)]
```

La compétence `LongAttack` hérite de la classe mère `Skill`, ce qui lui permet de profiter du système de cooldown commun. Elle possède un cooldown plus long que l'attaque au corps à corps, ce qui évite que le joueur puisse spammer les projectiles. Lorsque la compétence est disponible, sa méthode

`create_action()` est appelée. Elle change d'abord l'état d'animation du joueur :

```
caster.anim_status = "long range attack"
```

Cela permet de bloquer le joueur dans une animation d'attaque et de jouer la bonne spritesheet. Ensuite, la compétence crée une `ProjectileAction` contenant les informations nécessaires à la création du projectile :

```
ProjectileAction(caster.x, caster.y, caster.direction, True ou False)
```

Le booléen final permet de savoir si le projectile vient d'Aeden ou de Lyra. Ce choix est important car les deux personnages n'utilisent pas la même image de projectile. Dans la classe `Projectile`, l'image chargée dépend de ce booléen.

Le rôle de `ProjectileAction` est donc de transformer l'intention du joueur en objet de gameplay concret. Cette action reçoit la position du joueur, sa direction et l'identité du personnage. Elle corrige ensuite la direction afin qu'elle soit adaptée à la perspective isométrique.

Dans un jeu en vue de dessus classique, une direction diagonale (1, 1) peut être utilisée directement. Dans ce moteur, ce n'est pas le cas, car les diagonales visuelles de la grille isométrique ne correspondent pas aux diagonales mathématiques habituelles. Le code applique donc une correction :

(1, 1) devient (1, 1/2), (-1, 1) devient (-1, 1/2), (1, -1) devient (1, -1/2), (-1,-1) devient (-1,-1/2)

Cette transformation permet au projectile de suivre les lignes des tuiles isométriques. Sans cette correction, un tir diagonal paraîtrait trop vertical ou trop rapide par rapport au rendu de la carte. Ce choix rend le déplacement du projectile cohérent avec celui des joueurs, qui utilise la même logique de correction diagonale.

Une fois la direction corrigée, `ProjectileAction` crée réellement l'objet projectile :

```
self.projectile = Projectile(x, y, self.direction, aeden)
```

L'action ne déplace pas elle-même le projectile. Elle se contente de le créer, de jouer le son d'attaque, de l'envoyer au réseau si nécessaire, puis de l'ajouter dans la liste des projectiles actifs :

```
game.player_projectiles.append(self.projectile)
```

Cette liste est ensuite mise à jour à chaque frame dans la boucle principale du jeu grâce à :

```
context.update_player_projectiles(context.enemies)
```

La classe `Projectile` contient toute la logique physique du projectile. Elle stocke sa position (x, y), sa direction (dx, dy), sa vitesse, son image, sa hitbox et sa portée maximale. La vitesse est fixée à 5, ce qui signifie que le projectile avance de cinq unités par frame dans la direction donnée.

À chaque mise à jour, le projectile applique les formules suivantes :

```
x = x + direction_x * speed
```

```
y = y + direction_y * speed
```

Ces équations sont simples, mais elles suffisent à produire un mouvement fluide, car la direction a déjà été adaptée au repère isométrique. Par exemple, un projectile tiré en diagonale bas-droite avance avec un déplacement horizontal complet, mais seulement une demi-valeur verticale. Cela permet de respecter la pente visuelle des tuiles.

Après avoir modifié sa position, le projectile met à jour sa hitbox :

```
self.rect.x = self.x
```

```
self.rect.y = self.y - 32
```

Le décalage vertical -32 est nécessaire car la position logique du projectile ne correspond pas exactement au coin supérieur gauche de son image. Comme pour les joueurs, le sprite est affiché avec un offset afin d'être correctement aligné dans le monde isométrique. La hitbox doit donc être replacée pour correspondre à la zone réellement dangereuse du projectile.

Le projectile possède aussi une variable `range`, qui représente sa durée de vie. À chaque frame, cette valeur diminue :

```
self.range -= 1
```

Lorsque `range` atteint zéro, le projectile est supprimé. Ce système limite naturellement la distance maximale du tir. Plutôt que de calculer une distance parcourue avec une racine carrée, le moteur utilise ici un compteur de frames, ce qui est plus simple et moins coûteux. Avec une vitesse de 5 et une portée de

100, le projectile peut théoriquement parcourir environ 500 unités avant de disparaître, sauf s'il touche un obstacle avant.

La collision avec la carte repose sur la conversion inverse vers la matrice. À chaque frame, la position isométrique du projectile est convertie en coordonnées cartésiennes :

```
x_cart, y_cart = iso_to_cart_tile(self.x, self.y)
```

Le moteur regarde ensuite la couche haute de la case :

```
map_tiles[x_cart][y_cart][2]
```

Si cette couche n'est pas vide, cela signifie que le projectile rencontre un mur, une porte fermée ou un élément bloquant en hauteur. Dans ce cas, le projectile est supprimé :

```
return False
```

Ce choix est cohérent avec la structure 3D de la carte : le projectile peut traverser certaines zones au sol, mais il est bloqué par les éléments présents sur les couches supérieures.

Pour les collisions avec les ennemis, le système utilise les rectangles de Pygame. Chaque projectile possède un `pygame.Rect` de taille 40 x 40, centré autour de sa position logique. Les ennemis possèdent également une hitbox. La collision est donc testée avec :

```
self.rect.colliderect(target.hitbox)
```

Si une collision est détectée, le projectile applique ses effets :

```
target.take_damage(10, context)
```

```
target.stunned_countdown = 60
```

```
return False
```

L'ennemi reçoit donc 10 points de dégâts et est étourdi pendant 60 frames, ce qui correspond environ à une seconde si le jeu tourne autour de 60 FPS. Le projectile est ensuite détruit afin d'éviter qu'un même projectile touche plusieurs fois le même ennemi ou traverse toute une salle.

Le système prévoit aussi un cas particulier pour le boss final. Si `final_boss` existe, le projectile ne vérifie pas les collisions avec les ennemis classiques, mais avec la hitbox du boss :

```
self.check_collision_final_boss(final_boss)
```

Dans ce cas, le boss subit lui aussi 10 points de dégâts et un effet de stun est appliqué via

```
stunned_cooldown = 60.
```

Cette séparation permet d'éviter les confusions entre les ennemis normaux et le boss final, qui possède une logique propre.

L'affichage du projectile est intégré au rendu isométrique dans la classe `Map`. Le projectile n'est pas simplement dessiné par-dessus toute la scène. Sa position est d'abord reconverte en tuile avec `iso_to_cart_tile()`, puis il est affiché au bon moment dans le parcours de la matrice. Cette méthode permet de respecter l'ordre de profondeur isométrique. Le projectile peut ainsi apparaître correctement par rapport aux tuiles, aux joueurs et aux autres entités.

Le système est également compatible avec le multijoueur. Lorsqu'une `ProjectileAction` est créée localement, sa méthode `send_to_network()` envoie les informations nécessaires :

```
context.add_info_projectile(self.x, self.y, self.direction, self.aeden)
```

Le réseau ne transmet pas l'objet `Projectile` lui-même. Il transmet uniquement la position de départ, la direction corrigée et l'identité du personnage. Sur l'autre machine, `network.py` reçoit ces informations et reconstruit une nouvelle `ProjectileAction` avec `host=False`. Cette action recrée alors le même projectile localement, mais sans le renvoyer au réseau.

Cette logique permet aux deux joueurs de voir le projectile apparaître au même endroit, partir dans la même direction et utiliser la même image. Le fait d'envoyer uniquement les paramètres essentiels rend la synchronisation légère et évite de transmettre trop de données.

Le système des projectiles illustre donc bien l'architecture globale du jeu. Le joueur déclenche une compétence, la compétence crée une action, l'action crée un objet projectile, puis le projectile gère son déplacement, ses collisions et sa durée de vie. Chaque niveau a une responsabilité claire, ce qui rend le code plus lisible, plus modulaire et plus simple à maintenir.

2.2.7 Architecture du moteur de carte

2.2.7.1 Fonctions de reconnaissance RGB et génération de la carte

Le moteur isométrique utilise une image pixel art comme base complète de génération de la carte. Chaque pixel de cette image correspond à une case logique de la matrice du jeu. La couleur RGB du pixel permet ensuite de déterminer quel élément doit être placé à cette position.

Le système repose sur une série de fonctions de reconnaissance nommées `is_xxx(pixel)`. Chaque fonction vérifie les composantes rouge, verte et bleue du pixel afin d'identifier un type précis d'élément. Par exemple, un levier d'Aeden est reconnu avec la condition :

```
r == 187 and g == 135
```

tandis qu'un levier de Lyra est reconnu avec :

```
r == 90 and g == 105
```

Dans cette architecture, les composantes rouge et verte servent généralement à identifier la nature de l'objet : levier, porte, miroir, source lumineuse, récepteur, décoration, mur, sol, eau, arbre, ennemi, etc. La composante bleue est quant à elle principalement utilisée comme identifiant de groupe. Ce choix permet de lier plusieurs objets entre eux sans écrire cette relation directement dans le code.

Par exemple, deux leviers ayant la même valeur bleue appartiennent au même groupe d'énigme. Une porte ayant cette même valeur bleue pourra être ouverte lorsque l'énigme correspondante sera résolue. Le même principe est utilisé pour les puzzles de tuiles, les énigmes laser, les miroirs, les sources lumineuses et les récepteurs.

Ce principe est aussi utilisé pour les ennemis. Dans le fichier, les fonctions comme `is_slasher(pixel)`, `is_flinger(pixel)` ou `is_detonator(pixel)` reconnaissent le type d'ennemi à partir des valeurs rouge et verte. Ensuite, la valeur bleue est transmise lors de la création de l'ennemi :

```
context.add_slasher(x_iso, y_iso, pixel[2])
```

La valeur `pixel[2]` permet alors d'associer l'ennemi à un groupe, généralement lié à une salle. Cela rend possible une logique de progression par rooms : lorsque tous les ennemis d'un même groupe sont vaincus, les portes correspondant à ce groupe peuvent s'ouvrir automatiquement. La carte pixel art ne sert donc pas seulement à placer visuellement les ennemis, mais aussi à définir leur appartenance logique à une zone de combat.

2.2.7.2 Construction dynamique de la matrice

La fonction `image_to_matrix()` constitue le cœur de la génération de la carte. Elle commence par charger l'image avec Pillow, puis récupère sa largeur et sa hauteur. Ces dimensions servent directement à créer la matrice du jeu :

```
matrix = [[[0, 0, 0] for _ in range(height)] for _ in range(width)]
```

Chaque case contient trois valeurs, correspondant aux trois couches verticales de la carte. Cela permet de stocker par exemple un sol sur la première couche, un objet ou un mur sur la deuxième, puis un élément supérieur sur la troisième. Le moteur parcourt ensuite tous les pixels de l'image avec deux boucles imbriquées:

```
for y in range(height):
    for x in range(width):
        pixel = image.getpixel((x, y))
```

Pour chaque pixel, une suite de conditions ``if / elif`` détermine quel élément doit être créé. Lorsqu'un pixel correspond à un décor simple, la matrice reçoit seulement des identifiants de tuiles. Par exemple, une fleur, un arbre ou un mur ajoute uniquement des numéros dans les couches de la matrice.

En revanche, lorsqu'un pixel représente un objet interactif, le moteur crée aussi une instance Python et l'ajoute dans le `GameStateRegistry`. Par exemple, pour un levier d'Aeden, le moteur remplit la matrice avec les identifiants visuels du levier, puis crée un objet `Lever` :

```
gameRegistry.add_lever(Lever(True, x, y, pixel[2]))
```

Le booléen `True` indique que le levier appartient à Aeden. Les coordonnées `x` et `y` indiquent sa position dans la carte, et `pixel[2]` représente l'identifiant du groupe. Le même principe est utilisé pour les leviers de Lyra, avec un booléen ``False``.

Pour les artefacts, la valeur bleue permet de déduire leur identifiant :

```
id = pixel[2] - 160
```

Ainsi, les valeurs bleues de ``160`` à ``163`` correspondent aux quatre artefacts du jeu. Cette méthode évite d'avoir à coder manuellement chaque artefact : il suffit de placer le bon pixel dans l'image.

Les portes utilisent une logique plus détaillée. Elles sont reconnues avec une valeur rouge fixe et une valeur verte comprise entre 40 et 47. Cette valeur verte permet de déterminer l'orientation de la porte. Le moteur calcule ensuite :

```
tile_nb1 = 20 + g % 40
```

Cette valeur permet d'obtenir le sprite correspondant à l'orientation de la porte. Ensuite, le moteur construit une `orientation_list`, qui contient à la fois l'apparence de la porte et sa collision. Cela permet à une seule catégorie de pixels de représenter plusieurs orientations possibles.

Les énigmes laser utilisent également ce système. Un miroir d'Aeden, un miroir de Lyra, une source lumineuse et un récepteur sont reconnus par des couleurs différentes. Cependant, ils peuvent partager la

même valeur bleue afin d'appartenir à la même énigme laser. Lors de la lecture de l'image, le moteur crée automatiquement le `LaserPuzzle` correspondant puis lui ajoute les miroirs, sources et récepteurs.

2.2.7.3 Système d'actions et synchronisation

Le fichier `actions.py` met en place une architecture orientée objet qui sert de base à la majorité des interactions du jeu. L'idée principale est de représenter chaque événement important du gameplay sous la forme d'un objet `Action`. Cela permet de ne pas exécuter directement les conséquences d'une touche ou d'une interaction dans la boucle principale, mais de passer par une couche intermédiaire commune.

Toutes les actions héritent d'une classe mère `Action`, qui contient deux informations essentielles : le nom de l'action et un booléen `host`. Le nom permet d'identifier le type d'action à exécuter, notamment pour la synchronisation réseau, tandis que `host` indique si l'action a été créée localement ou si elle provient déjà du réseau.

Ce booléen est central dans le fonctionnement multijoueur. Lorsqu'un joueur crée une action localement, celle-ci doit être envoyée à l'autre joueur pour être reproduite sur sa machine. En revanche, lorsqu'un client reçoit une action depuis le réseau et la recrée localement, il ne faut surtout pas la renvoyer une seconde fois, sinon les deux clients se renverraient indéfiniment la même action. Le paramètre `host` permet donc d'éviter cette boucle.

La classe mère contient une méthode commune `send_to_network()` :

```
if self.host:
    game.action_created = True
    game.action_name_to_send.append(self.name)
```

Ce morceau de code indique au contexte de jeu qu'une action doit être partagée. Le nom de l'action est ajouté dans une liste d'actions à envoyer. Ensuite, chaque classe fille ajoute ses propres paramètres grâce à des méthodes spécifiques du contexte. Par exemple, `ProjectileAction` ajoute la position du projectile, sa direction et le personnage qui l'a tiré. Ainsi, le réseau n'envoie pas directement une instance Python complète, mais seulement les informations nécessaires pour reconstruire la même action chez l'autre joueur.

Cette architecture correspond au principe du Command Pattern. Une action est un objet qui encapsule une intention de gameplay : attaquer, interagir, tirer un projectile, activer un levier ou modifier une tuile. L'avantage est que la boucle principale du jeu n'a pas besoin de connaître tous les détails de chaque mécanique. Elle ajoute simplement des actions dans une file, puis le contexte les exécute dans l'ordre.

Ce système est directement relié à la classe `Player`. Lorsqu'un joueur utilise une compétence, il ne modifie pas lui-même le monde. La méthode `try_use()` du joueur appelle la compétence correspondante, et cette compétence crée une action. Par exemple, la compétence `SwordAttack` crée une `MeleeAction`, `LongAttack` crée une `ProjectileAction`, et `Interact` crée une `interactAction`. Le joueur déclenche donc une intention, puis l'action s'occupe de ses conséquences.

L'attaque au corps à corps est gérée par `MeleeAction`. Lorsqu'elle est créée localement, elle récupère directement la position et la direction du joueur. Lorsqu'elle est recrée depuis le réseau, elle peut recevoir ces informations sous forme de coordonnées et de direction. Cela permet à la même classe de fonctionner dans les deux cas.

L'action calcule ensuite une zone d'attaque devant le joueur à partir de sa position et de sa direction :

```
target_x = self.position[0] + 16 + dx * 32
target_y = self.position[1] - 16 + dy * 32
```

Ici, dx et dy représentent la direction du joueur. Le décalage dx * 32 et dy * 32 place la zone d'attaque devant le personnage, tandis que les valeurs +16 et -16 permettent d'ajuster la position de cette zone par rapport au sprite. Ensuite, un carré de collision est créé :

```
width = 50
attackRange = pygame.Rect(x - width//2, y - width//2, width, width)
```

Cette zone est comparée aux hitbox des ennemis avec `collidect()`. Si une collision est détectée, l'ennemi reçoit des dégâts avec `take_damage(20, game_context)`. Le même principe est appliqué au boss final si celui-ci est présent. L'action déclenche également une animation `SimpleSlashAnimation`, ce qui permet d'avoir un retour visuel immédiat au moment de l'attaque.

Le système d'interaction repose quant à lui sur `interactAction`. Cette action est particulièrement importante car elle centralise toutes les interactions avec l'environnement. Lorsqu'un joueur appuie sur la touche d'interaction, le jeu ne vérifie pas séparément les leviers, les tuiles, les miroirs ou les artefacts dans la boucle principale. Il crée une seule `interactAction`, qui se charge ensuite de déterminer quel objet est concerné.

L'action commence par convertir la position isométrique du joueur en coordonnées cartésiennes avec `iso_to_cart_tile()`. Cette conversion permet de retrouver la case exacte de la matrice sur laquelle le joueur se trouve. Ensuite, plusieurs méthodes de vérification sont appelées : `leverVerif()`, `tilePuzzleVerif()`, `artefactVerif()`, `mirrorVerif()` et `signVerif()`.

Chaque méthode parcourt une partie du `GameStateRegistry`. Par exemple, `leverVerif()` cherche si un levier se trouve à la position du joueur et vérifie aussi si ce levier appartient au bon personnage grâce à `lever.player == self.caster`. Cela permet de conserver la mécanique coopérative où certains objets sont réservés à Aeden ou à Lyra.

Si un levier est détecté, l'interaction ne modifie pas directement le levier. Elle ajoute une nouvelle action : `context.add_action(LeverAction(lever[0], lever[1]))`

Cela montre bien l'intérêt de l'architecture : même une interaction peut produire une autre action. Le système reste donc uniforme.

`LeverAction` reçoit l'identifiant du groupe et l'identifiant du levier. Lors de son exécution, elle récupère le levier dans le `GameStateRegistry`, vérifie qu'il n'est pas verrouillé, inverse son état avec `toggle(context.map)`, joue un son différent selon l'état du levier, puis appelle `gameRegister.check_open_door()`. Cette dernière vérification permet de savoir si l'énigme associée est terminée et si les portes doivent être ouvertes.

Le puzzle de tuiles fonctionne de manière proche avec `tilePuzzleAction`. L'action reçoit également un groupe et un identifiant de tuile. Elle vérifie si la tuile n'est pas verrouillée, joue un son, inverse l'état de la tuile, puis vérifie si le groupe de puzzle est terminé. L'intérêt est que les leviers et les tuiles suivent une logique commune : identifier l'objet, modifier son état, mettre à jour la matrice, vérifier l'ouverture des portes.

L'action `ProjectileAction` gère quant à elle l'attaque à distance. Elle reçoit la position du joueur, sa direction et le personnage qui tire. Comme pour les déplacements, la direction est corrigée pour correspondre à la perspective isométrique. Par exemple, une direction diagonale (1, 1) devient (1, 1/2). Cela permet au projectile de suivre visuellement les lignes de la grille isométrique au lieu de se déplacer comme dans une grille cartésienne classique.

Une fois la direction corrigée, l'action crée un objet `Projectile` :

```
self.projectile = Projectile(x, y, self.direction, aeden)
```

Lors de son exécution, elle joue le son du projectile, envoie les informations réseau si l'action est locale, puis ajoute le projectile à la liste `game.player_projectiles`. Le projectile sera ensuite mis à jour à chaque frame par la boucle principale.

L'intérêt de ce système est que la compétence du joueur, l'action réseau et l'objet projectile sont séparés. La compétence décide si le joueur a le droit de tirer, l'action crée le projectile et le synchronise, puis le projectile gère son propre déplacement et ses collisions.

Le système permet donc de structurer proprement les responsabilités. La classe `Player` ne contient pas toute la logique des attaques. Elle possède seulement une liste de compétences. Les compétences ne modifient pas directement le monde : elles créent des actions. Les actions appliquent les conséquences concrètes et peuvent être envoyées au réseau. Enfin, le contexte de jeu exécute les actions dans l'ordre.

Cette séparation est importante pour un jeu coopératif. Si chaque interaction modifiait directement la carte ou les ennemis depuis plusieurs endroits du code, il serait très difficile de garder les deux clients synchronisés. Avec ce système, chaque événement important est transformé en action identifiable, paramétrée et reproductible. Il suffit donc d'envoyer le nom de l'action et ses paramètres pour obtenir le même résultat chez l'autre joueur.

L'architecture d'actions permet aussi d'ajouter de nouvelles mécaniques sans réécrire la boucle principale. Pour ajouter une nouvelle interaction, il suffit de créer une nouvelle classe héritant de `Action`, de définir sa méthode `execute()`, puis de l'appeler depuis une compétence ou depuis `interactAction`. C'est ce qui rend le système extensible pour les futures énigmes, les nouveaux ennemis ou les nouvelles compétences.

2.2.7.4 Reconstruction des actions côté réseau

Le système réseau complète directement l'architecture des actions. Lorsqu'une action est créée localement, elle n'est pas envoyée sous forme d'objet Python, car une instance de classe ne peut pas être transmise directement de manière fiable entre deux machines. À la place, le jeu envoie un message JSON contenant le nom de l'action et les paramètres nécessaires à sa reconstruction.

Dans la boucle réseau, lorsque le message reçu possède :

```
data["msg"] == "action_created"
```

le jeu récupère la liste des actions envoyées dans `data["action"]`

Puis chaque nom d'action est testé pour reconstruire localement l'action correspondante. Par exemple, pour une attaque au corps à corps, le client distant recrée une `MeleeAction` en utilisant la position et la direction du joueur reçues dans `player_infos` :

```
game_context.add_action(MeleeAction(None, 32, data["player_infos"]["position"][0], data["player_infos"]["position"][1], data["player_infos"]["direction"][0], data["player_infos"]["direction"][1], host=False))
```

Le paramètre `caster` vaut ici `None`, car le joueur ayant créé l'action n'est pas l'objet local complet. On utilise donc directement les informations reçues par le réseau : position, direction et état du joueur. Le paramètre `host=False` indique que cette action vient du réseau et ne doit donc pas être renvoyée à l'autre client.

Le même principe est utilisé pour les projectiles. Lorsqu'une `ProjectileAction` est reçue, le réseau récupère la position de départ, la direction et le personnage qui a tiré :

```
action_data = ( data["info_action"]["Projectile coords"], data["info_action"]["Projectile direction"], data["info_action"]["Projectile de Aeden"])
```

Puis il reconstruit l'action :

```
game_context.add_action(actions.ProjectileAction(action_data[0][0], action_data[0][1], action_data[1], action_data[2], False))
```

Ainsi, le projectile est créé sur les deux machines avec les mêmes paramètres. Cela garantit que son affichage et sa trajectoire sont cohérents pour les deux joueurs.

Les interactions avec l'environnement suivent la même logique. Lorsqu'une `Interact Action` est reçue, le client distant récupère les coordonnées cartésiennes de l'interaction et le personnage qui l'a déclenchée :

```
action_data = (data["info_action"]["Interact coords"], data["info_action"]["Interact caster"])
```

Puis il ajoute une nouvelle `interactAction` dans sa propre file d'actions :

```
game_context.add_action(actions.interactAction(action_data[0][0], action_data[0][1], action_data[1], False))
```

Le fait de reconstruire l'action à partir des mêmes paramètres permet aux deux jeux d'exécuter exactement les mêmes conséquences : activation d'un levier, rotation d'un miroir, interaction avec une tuile, récupération d'un objet ou affichage d'un panneau.

Cette logique est également utilisée pour les attaques ennemies et le boss final. Le réseau vérifie le nom de l'action reçue, récupère les paramètres dans `info_action`, puis recrée l'action adaptée :

`SlasherAttack`, `FlingerAttack`, `DetonatorAttack` ou `FinalBossAttack`. Dans le cas des projectiles ennemis, le message réseau contient suffisamment d'informations pour reconstruire l'orbe : position, cible, vitesse et dégâts.

L'ensemble du système fonctionne donc comme une traduction entre objets Python et données JSON.

Localement, le jeu manipule des objets d'action riches et structurés. Sur le réseau, ces actions sont transformées en informations simples. À la réception, elles sont reconstruites sous forme d'objets et ajoutées à la file d'exécution du jeu.

2.2.7.5 Synchronisation régulière de l'état du jeu

En plus des actions ponctuelles, le réseau synchronise régulièrement l'état des entités. Un intervalle réseau est défini : `network_interval = 50`

Cela signifie que toutes les 50 millisecondes environ, le jeu envoie un paquet contenant les informations importantes du joueur local : `"player_infos": local_player.get_infos()`

La méthode `get_infos()` du joueur fournit notamment : la position ; la direction ; l'état de déplacement ; l'état de sprint ; la hitbox ; l'animation actuelle ; l'état d'obscurité ; les points de vie.

À la réception, ces informations sont utilisées pour mettre à jour le joueur distant :

```
distant_player.x = data["player_infos"]["position"][0]
distant_player.y = data["player_infos"]["position"][1]
distant_player.direction = tuple(data["player_infos"]["direction"])
distant_player.is_moving = data["player_infos"]["is_moving"]
distant_player.anim_status = data["player_infos"]["anim_status"]
```

Ce fonctionnement permet de séparer deux types de synchronisation :

les actions, qui représentent des événements ponctuels ;

l'état des entités, qui est envoyé régulièrement.

Cette séparation est importante. Les actions servent à reproduire les conséquences du gameplay, tandis que les informations régulières permettent de garder les positions, animations et points de vie cohérents entre les deux joueurs.

2.2.7.6 Synchronisation des ennemis et du boss

Le code réseau prévoit aussi une logique d'autorité côté host. Si le joueur local est l'hôte, il envoie l'état des ennemis à l'autre joueur :

```
for ennemi in game_context.enemies:
    if ennemi != None:
        key, vals = ennemi.state_info()
        data_to_send["enemies"][key] = vals
```

Cela signifie que l'intelligence artificielle des ennemis est principalement contrôlée par l'hôte. Le client distant ne recalcule pas l'IA de son côté ; il reçoit les positions, directions, points de vie, hitbox et états d'IA envoyés par l'hôte.

À la réception, le client met à jour ses ennemis locaux :

```
game_context.enemies[enemy_id].x = infos["position"][0]
game_context.enemies[enemy_id].y = infos["position"][1]
game_context.enemies[enemy_id].hp = infos["hp"]
game_context.enemies[enemy_id].AI_state = infos["AI_state"]
```

Cette organisation évite que les deux machines calculent une IA légèrement différente, ce qui pourrait provoquer une désynchronisation. Le host devient donc la référence pour les ennemis.

Le même principe est appliqué au boss final. Si le boss existe, le host envoie sa position, ses points de vie et son état d'IA. L'autre joueur reçoit ces informations et met à jour son boss local.

Cette architecture permet de garder un gameplay cohérent en multijoueur tout en limitant les conflits entre les deux simulations.

2.2.8 Fenêtre d'introduction au lancement d'une partie

Une capture d'écran de la fenêtre est disponible en Annexe 2.2.8

J'ai implémenté dans `game_logic.py` une classe `IntroWindow`, dont le rôle est d'afficher une fenêtre narrative au-dessus du jeu lorsqu'une partie commence. Cette fenêtre apparaît avant que le joueur puisse réellement contrôler son personnage, afin d'introduire le contexte du jeu et d'expliquer l'objectif général de l'aventure.

La classe repose sur une surface transparente créée avec Pygame :

```
self.surface = pygame.Surface((1, 1), pygame.SRCALPHA)
```

Cette surface est ensuite redimensionnée dans `update_layout()` pour prendre la taille réelle de l'écran logique du jeu. Cela permet d'afficher l'introduction directement dans le même espace que le gameplay, avant le redimensionnement final de la fenêtre.

Le texte d'introduction est stocké directement dans la classe sous forme de chaîne multiligne :

```
self.text = ("Au coeur du sanctuaire se dressent quatre voies  
oubliées,\n", "chacune abritant un artefact ancien imprégné de lumière et  
d'ombre.\n\n", ...)
```

Ce texte permet de présenter les quatre branches du jeu, les artefacts à récupérer, les énigmes environnementales et les ennemis à affronter.

L'affichage utilise un système de fondu avec la variable `alpha` et l'état `fade_state`. Au lancement, `start()` initialise la fenêtre :

```
self.alpha = 0  
self.fade_state = "fade in"  
self.update_layout()
```

Puis la méthode `update()` augmente progressivement l'opacité jusqu'à rendre la fenêtre visible.

Lorsqu'elle doit disparaître, l'opacité diminue jusqu'à zéro, puis `context.intro_done` passe à `True`, ce qui permet au jeu de commencer réellement.

La méthode `draw()` s'occupe du rendu. Elle affiche un grand rectangle noir semi-transparent au centre de l'écran, puis dessine chaque ligne du texte avec une police spécifique :

```
font = pygame.font.Font("assets/fonts/CormorantGaramond-semibold.ttf", h //  
22)
```

Une croix est aussi dessinée en haut à droite de la fenêtre afin de permettre au joueur de fermer l'introduction.

Dans `update_game()`, tant que `context.intro_done` vaut `False`, le jeu met uniquement à jour et affiche cette fenêtre, puis quitte la fonction avec `return`. Cela bloque temporairement le gameplay pendant l'introduction.

Cette classe permet donc d'ajouter une vraie entrée narrative au jeu sans créer un écran séparé.

L'introduction est dessinée directement au-dessus du jeu, avec un fondu progressif et une fermeture interactive, tout en restant intégrée au système d'affichage principal.

2.2.9 Choix final et fins coopératives

Pour conclure le jeu, j'ai développé un système de choix final coopératif entièrement intégré dans `game_logic.py`. Après la mort du boss final, les deux joueurs doivent prendre une décision importante qui détermine la fin obtenue. Cette mécanique permet de terminer le jeu sur une dernière interaction multijoueur cohérente avec tout le principe de coopération asymétrique utilisé dans le projet.

Une capture d'écran du choix coopératif est présentée en annexe 2.2.9

Le système commence lorsque le boss final meurt. Dans `update_game()`, lorsque les points de vie du boss atteignent zéro, son état passe à `"DEAD"` :

```
if context.final_boss and context.final_boss.hp <= 0:  
context.final_boss.AI_state = "DEAD"
```

Une animation de disparition est ensuite jouée grâce à `time_before_disappear`. Une fois cette animation terminée, le moteur active l'état : `context.final_state = "CHOICE"`

À partir de ce moment, le jeu entre dans une nouvelle phase où les déplacements, les combats et les interactions classiques sont remplacés par une interface de décision finale.

L'interface du choix est dessinée dans la fonction `draw_final_choice_ui(playerL)`. Cette fonction crée d'abord une surface transparente affichée au-dessus du jeu : `overlay =`

```
pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
overlay.fill((10, 10, 10, 170))
```

Ce voile semi-transparent permet de conserver le décor du jeu visible en arrière-plan tout en assombrissant l'écran afin de focaliser l'attention du joueur sur la décision finale. J'ai voulu conserver le rendu du monde jusqu'au bout plutôt que de changer brutalement d'écran.

Deux boutons sont ensuite générés dynamiquement avec `pygame.Rect` :

```
sacrifice_rect = pygame.Rect(...)
live_rect = pygame.Rect(...)
```

Le premier bouton correspond au sacrifice, tandis que le second représente la survie. Les deux choix utilisent volontairement une opposition visuelle très marquée : le sacrifice utilise un thème clair alors que la survie utilise un thème sombre. Cela renforce le contraste symbolique entre les deux décisions.

Les rectangles sont ensuite stockés dans le contexte :

```
context._sacrifice_rect = sacrifice_rect
context._live_rect = live_rect
```

Cette étape est importante car elle permet ensuite à `update_game()` de détecter les clics du joueur avec :

```
context._sacrifice_rect.collidepoint(mouse_pos)
```

Le système de choix est donc directement relié au système d'événements Pygame utilisé dans le reste du moteur.

Lorsqu'un joueur clique sur une option, plusieurs opérations sont réalisées :

```
context.player_choice = "sacrifice"
send_choice("sacrifice")
context.final_state = "RESOLVE"
```

Le choix local est d'abord sauvegardé dans `player_choice`. Ensuite, la fonction `send_choice()` transmet cette information au second joueur grâce au système réseau déjà utilisé pour les actions du jeu.

Le choix final n'a pas été implémenté comme une véritable Action orientée objet contrairement aux attaques ou aux interactions. J'ai préféré utiliser un échange simplifié via les données réseau existantes :

```
if "choice" in action: game_context.remote_choice =
data["info_action"]["choice"]
```

Cette logique permet simplement de synchroniser la décision des deux joueurs sans créer une nouvelle classe d'action complète. Le réseau transporte donc ici une information spéciale liée à l'état final du jeu.

Une fois les deux choix reçus, la fonction `check_final_resolution()` décide de la fin obtenue :

```
if (context.player_choice == "sacrifice" and context.remote_choice ==
"sacrifice"):
    context.final_result = "ASCENSION"
else:
    context.final_result = "FALL"
```

Le fonctionnement est volontairement strict : la bonne fin n'est obtenue que si les deux joueurs acceptent ensemble le sacrifice. Si un seul joueur refuse, le résultat devient "FALL".

Ce système renforce directement le thème coopératif du jeu. La réussite finale ne dépend pas des compétences mécaniques du joueur mais de la capacité des deux joueurs à prendre une décision commune. La coopération ne concerne donc plus seulement les énigmes ou les combats, mais également le scénario lui-même.

Une fois la décision calculée, l'état du jeu passe à :

```
context.final_state = "RESULT"
```

Le moteur affiche alors l'écran de fin avec `draw_final_result()`. Cette fonction utilise un système de fade progressif pour faire apparaître les textes ligne par ligne :

```
alpha = min(255, (context.final_result_timer - appear_time) * 4)
```

Chaque phrase apparaît donc progressivement avec un délai entre les lignes, ce qui crée une mise en scène plus cinématique. La bonne fin utilise un fond blanc lumineux alors que la mauvaise fin utilise un fond noir sombre, afin de prolonger l'opposition lumière / obscurité présente dans tout le jeu.

Enfin, lorsque le joueur appuie sur espace, le moteur affiche les crédits grâce à :

```
context.credit_on = True
```

Toute cette architecture repose donc sur plusieurs états successifs (CHOICE, RESOLVE, RESULT) qui permettent de transformer progressivement le gameplay classique en séquence narrative finale sans changer complètement de boucle de jeu.

2.2.10 Bilan personnel et apport au projet

Mes contributions ne se limitent pas au moteur isométrique. J'ai également conçu une grande partie de l'architecture logicielle du projet : système de sauvegarde, système d'actions, architecture des compétences, interactions coopératives, puzzles complexes, moteur de projectiles, lecture de cartes par pixel art et gestion avancée des collisions.

Ce qui ressort principalement du code est la volonté de garder un système modulaire. Les objets du monde sont stockés dans un registre central, les interactions passent par des actions, les compétences créent ces actions, et le moteur isométrique se charge uniquement d'interpréter la matrice pour produire un affichage cohérent.

L'ensemble de ces choix techniques permet d'obtenir un jeu coopératif plus robuste, plus facilement extensible et mieux adapté à une architecture multijoueur.

Ressources

- <https://medium.com/%40aliounebfall/quest-fantasy-devlog-1-d35e71b18182>
- <https://docs.python.org/3/tutorial/classes.html>
- <https://realpython.com/python3-object-oriented-programming/>
- https://www.reddit.com/r/gamedev/comments/v2duc7/how_isometric_coordinates_work_in_2d_games/?tl=fr
- https://www.reddit.com/r/gamemaker/comments/8k9fqg/intro_to_isometric_projection/?tl=fr
- <https://www.youtube.com/watch?v=2Ucv9XM0pFI>
- <https://fr.wikipedia.org/wiki/Isom%C3%A9trie>

2.3. Antoine MUH

2.3.1 Présentation générale de mon rôle dans le projet

Dans le projet *Echoes Of Light*, j'ai occupé un rôle à la fois technique et artistique. J'ai principalement travaillé sur deux aspects du jeu : le développement du menu principal multijoueur et la création graphique des personnages ainsi que de plusieurs ennemis.

Cette double responsabilité m'a amené à travailler aussi bien sur la programmation que sur le pixel art et l'animation. Pour la partie technique, j'ai développé les différentes interfaces du jeu avec Python et la bibliothèque Pygame. Mon travail consistait à créer les menus, gérer les interactions des joueurs et assurer la communication avec le système réseau développé par les autres membres de l'équipe.

En parallèle, j'ai participé à la direction artistique du projet en réalisant les sprites et animations du jeu. J'ai notamment créé les personnages principaux ainsi que plusieurs ennemis en respectant les contraintes de la vue isométrique et du style pixel art choisi pour le projet.

Tout au long du projet, j'ai également travaillé en collaboration avec les autres membres de l'équipe afin de garantir la compatibilité entre les différents systèmes du jeu, aussi bien au niveau du réseau que des animations ou de l'interface utilisateur.

2.3.2 Développement du menu principal multijoueur

2.3.2.1 Conception de l'interface utilisateur

L'une de mes premières missions dans le projet a été de concevoir et développer le menu principal du jeu. Ce menu représentait une partie importante du projet puisqu'il constituait le premier élément visible par les joueurs. Il devait donc être clair, simple à utiliser et cohérent avec l'univers visuel du jeu.

Comme *Echoes Of Light* est un jeu multijoueur fonctionnant avec un système de sessions et de codes de connexion, le menu devait permettre aux joueurs de créer une partie ou d'en rejoindre une facilement. J'ai donc développé plusieurs écrans différents avec Python et Pygame, notamment un écran principal, un menu de création de session ainsi qu'un système permettant d'entrer un code pour rejoindre une partie.

Le développement de cette interface ne concernait pas uniquement l'aspect visuel. J'ai aussi dû réfléchir à l'organisation des informations et à la manière dont les joueurs allaient interagir avec les différents menus. L'objectif était de rendre la navigation intuitive afin que les joueurs puissent accéder rapidement aux fonctionnalités du jeu.

J'ai également travaillé l'aspect graphique du menu afin qu'il reste cohérent avec la direction artistique générale du projet. Les couleurs, les polices et les différents éléments visuels ont été choisis pour s'intégrer naturellement à l'univers pixel art d'*Echoes Of Light*. (voir annexe 2.3.1 pour la première version du menu)

2.3.2.2 Gestion du système multijoueur depuis le menu

Le menu devait aussi communiquer directement avec le système réseau du jeu. Lorsqu'un joueur créait une partie, le menu récupérait automatiquement le code de session généré puis l'affichait à l'écran afin qu'il puisse être partagé avec les autres joueurs.

Pour permettre à un joueur de rejoindre une partie, j'ai développé un système de saisie de texte dans Pygame. Comme cette bibliothèque ne propose pas de champ de texte intégré, j'ai dû gérer manuellement les entrées clavier, l'affichage des caractères et la validation des informations saisies.

De nombreux tests ont été réalisés afin de vérifier le bon fonctionnement du système et d'assurer une bonne communication entre l'interface utilisateur et les modules réseau du jeu.

2.3.2.3 Difficultés rencontrées et améliorations

Le développement du menu principal a nécessité plusieurs phases de correction et d'amélioration. Les premières versions présentaient différents problèmes techniques, notamment des boutons qui répondaient mal, des éléments qui se chevauchaient ou encore des transitions incorrectes entre certains écrans.

Le projet évoluant constamment, j'ai aussi dû adapter régulièrement le menu afin d'intégrer de nouvelles fonctionnalités. Certaines parties du code ont dû être réorganisées afin de conserver une interface claire et facile à maintenir malgré les modifications successives.

La collaboration avec les autres membres de l'équipe était également importante. Les changements effectués sur le système réseau ou sur certaines mécaniques du jeu avaient parfois un impact direct sur le fonctionnement du menu. Il fallait donc ajuster régulièrement l'interface afin de conserver la compatibilité entre les différents systèmes du projet.

Ces différentes phases de correction et de tests ont permis d'obtenir progressivement un menu plus stable, plus fluide et plus agréable à utiliser.

2.3.2.4 Ajout du système de sauvegarde

Après la création de la première version stable du menu, nous avons rajouté un système de chargement de sauvegarde directement accessible depuis l'interface principale.

Cette fonctionnalité permettait aux joueurs de reprendre une partie déjà commencée sans devoir recréer une session depuis le début. Son intégration a demandé plusieurs modifications au niveau de la navigation et de l'organisation des menus afin de conserver une interface simple et intuitive.

Pendant toute la durée du projet, le menu a continué à évoluer. J'ai apporté différentes améliorations visuelles et techniques afin d'améliorer la lisibilité, la fluidité des transitions et la stabilité générale de l'interface.

La version finale du menu (annexe 2.3.2) était donc beaucoup plus aboutie que les premières versions développées au début du projet.

2.3.3 Travail graphique et artistique

2.3.3.1 Création des personnages jouables

En parallèle du développement du menu, j'ai réalisé une grande partie du travail graphique du projet. J'ai notamment créé les deux personnages principaux du jeu : Aeden et Lyra.

Pour chacun de ces personnages, j'ai conçu leur apparence, leurs sprites et l'ensemble de leurs animations. Le jeu utilisant une vue isométrique en pixel art, chaque personnage devait être représenté

dans huit directions différentes afin de rester cohérent pendant les déplacements et pour un rendu plus visuel, chaque direction comportait huit frames.

Ce travail représentait une charge importante car toutes les animations devaient être dessinées image par image. Pour faire cela, j'ai utilisé [Pixellab.ai](#) (annexe 2.3.3), un outil d'intelligence artificielle qui m'a permis de générer une base pour un grand nombre de mes designs. Ensuite, j'ai utilisé Pixellorama (annexe 2.3.4) qui est un outil de création et d'édition de pixel art pour créer ou perfectionner ces dessins. Pour chaque personnage et ennemi, j'ai réalisé les animations de déplacement et de leurs différentes attaques. Enfin, j'ai rassemblé ces différents dessins en spritesheets utilisant [codeshack.io](#) (annexe 2.3.5).

L'objectif principal était de rendre les personnages immédiatement reconnaissables grâce à leur design et à leur style de combat.

2.3.3.2 Aeden : le gardien de la lumière

Aeden est un personnage associé au thème de la lumière. Pour son design, j'ai utilisé principalement des couleurs claires et une silhouette imposante afin de transmettre une impression de puissance et de maîtrise. (voir annexe 2.3.6)

Ses capacités sont directement liées à la lumière. Il peut activer certains mécanismes présents dans les niveaux et révéler des éléments cachés grâce à ses pouvoirs. J'ai donc travaillé les effets visuels de ses attaques afin qu'ils restent lisibles malgré les contraintes du pixel art.

Aeden utilise également une grande épée appelée l'Espadon des Lumières. J'ai créé ses animations de combat pour transmettre une impression de puissance tout en gardant des mouvements fluides et compréhensibles pour le joueur.

2.3.3.3 Lyra : la maîtresse des ombres

Lyra représente le thème des ombres et sert de contraste visuel à Aeden. Son design repose sur des couleurs plus sombres et une silhouette plus discrète afin de mettre en avant son agilité et sa furtivité. (annexe 2.3.7)

Ses pouvoirs lui permettent de manipuler des structures d'ombre présentes dans les niveaux. J'ai travaillé les animations et les effets visuels de ses capacités afin que le joueur comprenne facilement quelles structures peuvent être utilisées.

Son arme, la Lame des Ombres, possède un style différent de celui d'Aeden. J'ai donc créé des animations plus sombres afin de différencier clairement les deux personnages.

2.3.3.4 Production des spritesheets et contraintes techniques

La création des spritesheets représentait une partie très importante du travail graphique. J'ai réalisé toutes les animations nécessaires au fonctionnement du gameplay en utilisant le site [codeshack.io](#) (annexe 2.3.5).

Chaque personnage disposait d'animations de déplacement, d'attaques à distance, d'attaques au corps-à-corps, d'attaques de zones ou encore d'animations d'interactions.

Les spritesheets devaient également respecter une organisation très précise afin d'être compatibles avec le moteur du jeu. Les frames devaient être placées dans le bon ordre et respecter les dimensions attendues par le système d'animation.

Pendant la production, j'ai effectué plusieurs corrections afin d'améliorer la fluidité des animations et de corriger certains problèmes de proportions ou de cohérence entre les différentes directions des personnages. (annexe 2.3.8)

2.3.4 Création des ennemis et du bestiaire

2.3.4.1 Conception du bestiaire

En plus des personnages jouables, j'ai créé plusieurs ennemis principaux du jeu. Mon objectif était de proposer des ennemis visuellement différents afin que le joueur puisse reconnaître rapidement leur comportement pendant les combats.

Chaque ennemi devait avoir une silhouette identifiable et des animations permettant de comprendre immédiatement son rôle dans le gameplay.

2.3.4.2 Le Flinger

Le premier ennemi que j'ai créé était spécialisé dans les attaques à distance. Son apparence mettait en avant son rôle grâce à son style de rôdeur cagoulé. (annexe 2.3.9)

J'ai travaillé les animations de lancer afin que le joueur puisse anticiper les attaques. Les déplacements ont également été animés de manière à transmettre une impression d'agilité et de mobilité.

Toutes les animations de cet ennemi devaient aussi fonctionner correctement avec le système d'intelligence artificielle développé par les autres membres de l'équipe.

2.3.4.3 Le Slasher

Le second ennemi conçu était un chevalier noir spécialisé dans le combat au corps-à-corps. Son design reposait sur une armure lourde, une grande épée et une silhouette imposante afin de transmettre une sensation de danger immédiat. (annexe 2.3.10)

Les animations de combat ont demandé un travail important afin de rendre les attaques puissantes et lisibles. Chaque mouvement devait permettre au joueur de comprendre le timing des attaques et de réagir correctement pendant les combats.

La représentation de la grande épée en mouvement était particulièrement complexe à réaliser en pixel art isométrique, ce qui a nécessité plusieurs essais avant d'obtenir un résultat satisfaisant.

2.3.4.4 Le Detonator

Le troisième ennemi que j'ai développé était une sorcière capable d'infliger des dégâts de zone. Son apparence devait transmettre une impression de magie et de menace surnaturelle. (annexe 2.3.11)

J'ai créé les effets visuels de ses attaques afin qu'ils soient facilement identifiables et différents des pouvoirs des personnages jouables. Les animations devaient permettre au joueur de comprendre rapidement quelles zones représentaient un danger pendant les combats.

Ce travail demandait de conserver un bon équilibre entre lisibilité et cohérence avec le style graphique général du jeu.

2.3.5 Finitions visuelles et intégration dans le moteur du jeu

2.3.5.1 Travail de finition

Enfin, j'ai travaillé les transitions entre les animations afin d'éviter des changements trop brusques qui auraient donné un rendu moins naturel pendant les phases de jeu. Chaque animation devait revenir le plus proche possible de la position initiale du joueur pour un meilleur visuel.

2.3.5.2 Intégration technique et collaboration

L'intégration des sprites et animations dans le moteur du jeu nécessitait une collaboration régulière avec les autres membres de l'équipe. Les fichiers devaient respecter plusieurs contraintes techniques concernant les dimensions, l'organisation des frames et les formats utilisés.

Des tests réguliers ont permis d'identifier différents problèmes comme des erreurs d'alignement, des désynchronisations ou des animations incorrectes. J'ai ensuite corrigé ces problèmes afin d'assurer un fonctionnement stable dans le jeu final.

Ce travail m'a également permis de mieux comprendre les contraintes techniques liées au moteur du jeu et d'adapter mes productions en conséquence.

2.3.6 Bilan personnel et apport au projet

Le travail que j'ai réalisé sur *Echoes Of Light* a contribué à deux aspects importants du projet : l'interface utilisateur et l'identité visuelle du jeu.

Le développement du menu principal a permis de rendre le système réseau accessible et simple à utiliser pour les joueurs. De son côté, le travail graphique a donné une identité visuelle cohérente aux personnages, aux ennemis et aux différentes animations du jeu.

Ce projet m'a également permis de développer plusieurs compétences techniques et artistiques. J'ai renforcé ma maîtrise de Python et de Pygame, tout en développant des compétences en pixel art, animation et conception visuelle.

Enfin, cette expérience m'a permis de découvrir concrètement le fonctionnement d'un projet de jeu vidéo en équipe, notamment en ce qui concerne la communication, l'intégration technique et l'organisation du travail collaboratif.

Ressources :

- <https://www.pixellab.ai/create-character>
- <https://pixelorama.org/>
- <https://codeshack.io/images-sprite-sheet-generator/>
- <https://www.youtube.com/watch?v=iJ7DDqaGbG4>

2.4. Gustave SCHWIEN :

Au cours de ces mois de développement de notre jeu *Echoes of Lights*, je me suis principalement concentré sur le moteur logique du jeu, à savoir le gameplay, la conception des ennemis et de leur intelligence artificielle (IA) et des algorithmes de *pathfinding*, la conception des énigmes et l'implémentation des salles de combat, l'intégration de la musique et des effets audio, et enfin, l'implémentation du boss final.

2.4.1. Conception générale et game design

Avant d'entamer la phase de programmation pure, il a été nécessaire de définir les règles du jeu et la manière dont les joueurs allaient interagir avec l'environnement. Mon rôle a été central dans la logique globale du jeu et dans son fonctionnement.

2.4.1.1. Les personnages des joueurs : Aeden et Lyra

Pour que la coopération soit au cœur de l'expérience, il fallait éviter que les joueurs aient des rôles interchangeables. J'ai donc participé à la conceptualisation des capacités de nos deux personnages principaux :

- **Aeden** : il possède l'épée de lumière, arme lui permettant de combattre au corps à corps. Il est aussi capable de créer des projectiles de lumière lui permettant d'infliger des dégâts et d'étourdir les ennemis. Il possède également un flash de lumière, qui lui permet d'interagir avec les vestiges du sanctuaire.
- **Lyra** : coéquipière d'Aeden, elle possède la lame des ombres, épée capable d'anéantir tout ennemi qui la rencontre. À cela s'ajoute un orbe d'ombre qu'elle peut lancer, capable de stopper temporairement la course d'un ennemi. Enfin elle est capable d'interagir avec les objets des ombres, compétence nécessaire à la résolution des énigmes du jeu.

Chaque personnage est donc capable d'attaquer de près à l'épée, et de loin grâce à leurs projectiles. Leurs compétences d'interaction respectives leur permettent également de modifier leur environnement.

2.4.1.2. Conception des énigmes

L'aspect "réflexion" d'*Echoes of Lights* a nécessité la conception d'une série d'énigmes permettant de briser la monotonie des phases de combat. Notre ambition initiale était d'en créer un grand nombre, mais la réalité du développement nous a poussés à privilégier la qualité sur la quantité. J'ai conceptualisé le scénario et la logique de trois énigmes distinctes qui s'intègrent parfaitement à la grille isométrique :

- **L'énigme des leviers** : des mécanismes activables qui modifient l'état d'autres éléments de la salle, exigeant une logique combinatoire. Aeden contrôle les leviers de lumière, Lyra ceux de l'ombre.
- **Le puzzle de tuiles** : un défi spatial où les joueurs doivent interagir avec des dalles en leur faisant passer de l'ombre à la lumière, et inversement. Lyra peut interagir avec les tuiles des ombres, et Aeden avec les tuiles de lumière. Lorsqu'une tuile change d'état, les tuiles voisines changent également, ce qui rend le puzzle particulièrement complexe.
- **L'énigme des lasers** : un puzzle basé sur des émetteurs et des récepteurs nécessitant de tourner les miroirs de lumière et d'ombre pour dévier les lasers et ainsi les diriger vers le récepteur.
- **Le labyrinthe des ombres** : un labyrinthe auquel seule Lyra peut accéder. Seulement, il y fait sombre : Lyra ne peut donc pas se repérer seule à l'intérieur. Aeden, lui, est capable d'y voir grâce à sa lumière. Lyra doit donc être guidée par Aeden pour trouver la sortie.

2.4.1.3. Les salles de combat

Outre les énigmes, la progression s'articule autour de salles de combat dont j'ai défini l'architecture et la logique de validation. Mon objectif était de construire une courbe de difficulté organique. J'ai joué sur trois paramètres majeurs :

- **L'architecture de la salle** : l'ajout de murs, d'arbres, et d'autres obstacles permettant aux joueurs mais aussi aux ennemis de se protéger.
- **La densité et la disposition** : placer les ennemis en essaim pour inciter à l'utilisation d'attaques de zone, ou les espacer pour diviser la force de frappe des joueurs.
- **La composition des groupes** : mélanger des ennemis de mêlée et à distance pour créer des situations de stress tactique.

D'un point de vue technique, j'ai mis en place un système de `group_id`. Chaque ennemi généré sur la carte se voit attribuer un identifiant numérique correspondant à sa salle (son groupe). À chaque *frame*, le jeu scrute la liste `context.enemies`. Si la fonction détecte que tous les ennemis possédant un `group_id` spécifique ont été éliminés, elle déclenche automatiquement l'ouverture de la porte bloquant l'accès à la salle suivante. Ce système modulaire, couplé à notre génération de carte par fichier PNG pixel par pixel, nous a permis de *designer* de nouvelles salles rapidement sans modifier le code source.

2.4.2. L'Intelligence Artificielle (IA)

Le développement de l'IA a été la tâche la plus complexe de mon périmètre. Il a fallu créer un système capable de gérer de multiples entités simultanément, de calculer des chemins dans un environnement isométrique et de prendre des décisions cohérentes.

2.4.2.1. La classe `Enemy`

Pour garantir un code propre et évolutif, j'ai conçu une classe globale `Enemy` qui hérite de la classe mère `Entity`. Cette hiérarchie est cruciale : la classe `Entity` gère les coordonnées, la santé (`hp`) et la vitesse, tandis que la classe `Enemy` centralise le "cerveau" commun à tous les monstres. Ainsi, pour créer un nouvel adversaire, il suffit de créer une sous-classe (ex: `class Slasher(Enemy)`) qui hérite automatiquement de toutes les capacités de déplacement et de détection, me laissant uniquement le soin de rajouter la méthode d'attaque spécifique.

La fonction principale de l'IA est la méthode `update()`. Appelée à chaque boucle du jeu, elle agit comme une machine à états. Elle analyse la distance relative entre l'ennemi et les deux joueurs, puis détermine l'état actif de l'IA :

- **IDLE** : L'ennemi est inactif ou en attente.
- **CHASE** : Le joueur entre dans le rayon de détection, l'ennemi recalcule son chemin pour s'approcher à l'aide de son algorithme de pathfinding sur lequel nous reviendrons.
- **ATTACK** : Le joueur est à portée, l'ennemi attaque.

2.4.2.2. Les animations directionnelles

La vue isométrique implique de rendre les mouvements naturels dans 8 directions distinctes (Nord, Sud, Est, Ouest, et les quatre diagonales). Pour gérer cela, j'ai implémenté la fonction `update_facing()`. En calculant le vecteur de déplacement de l'ennemi (s'il va plus vers la gauche, vers le bas, etc...), la fonction détermine son orientation actuelle.

Parallèlement, j'ai mis en place la fonction `update_animation()` qui gère les *spritesheets* (feuilles de sprites). Chaque ennemi possède trois dictionnaires d'images (Marche, Attaque, Inactif). Chaque dictionnaire contient les 8 directions, et chaque direction contient une liste de *frames* (sauf la dernière puisque c'est un état où l'ennemi ne bouge pas). À chaque tour de boucle, le code incrémente un `frame_index` et met à jour l'image affichée (`self.image`). J'ai dû implémenter des sécurités mathématiques (les modulus `% len(frames)`) pour que l'animation boucle sans jamais provoquer d'erreur d'index.

2.4.2.3. Gestion des hitboxes et mécanique de "stun"

La gestion des collisions s'opère via les `pygame.Rect`. J'ai dû lier ces rectangles invisibles aux coordonnées réelles des ennemis pour qu'ils les suivent parfaitement, permettant ainsi au moteur de jeu de calculer les collisions avec les murs et les attaques des joueurs.

J'ai également intégré une mécanique d'étourdissement (*stun*). J'ai ajouté un attribut `stunned_countdown` initialisé à 0. Lorsqu'un joueur touche un ennemi avec son projectile, ce compteur passe à une valeur positive (ex: 60 frames). Dans la méthode `update()`, la toute première condition vérifie ce compteur. S'il est supérieur à 0, l'ennemi décrémente la valeur et la méthode retourne `None` immédiatement, coupant court à toute autre action (déplacement, attaque). L'ennemi reste alors totalement figé visuellement et logiquement jusqu'à la fin de l'étourdissement.

2.4.2.4. Le pathfinding : implémentation de l'algorithme A* (A-Star)

Pour que les ennemis traquent les joueurs de manière intelligente en contournant les obstacles, j'ai développé mon propre algorithme A*. Cet algorithme de recherche de chemin utilise une heuristique mathématique (la distance de Tchebychev légèrement modifiée pour l'adapter au jeu) afin de trouver le chemin le plus court sur la matrice du jeu.

L'algorithme place la tuile de départ et d'arrivée dans une file de priorité (`heapq`). Il explore les tuiles adjacentes en appelant constamment ma fonction `iswalkable()`, qui lit la matrice de la carte en

coordonnées cartésiennes et renvoie `True` si la tuile est libre ou `False` si c'est un obstacle (tuile non franchissable).

Une fois la cible trouvée, l'algorithme remonte les nœuds parents pour construire une liste de coordonnées. L'ennemi suit alors cette liste point par point via la fonction `chase()`.

2.4.2.5. Algorithme de Bresenham pour `iswalkable_line()`

Un problème majeur est rapidement apparu : les ennemis à distance tentaient de tirer à travers les murs s'ils étaient suffisamment proches du joueur. Il fallait que les ennemis aient une ligne de vue. J'ai implémenté l'algorithme de Bresenham, une formule mathématique permettant de tracer une ligne parfaite sur une grille discrète (pixel par pixel, ou ici, tuile par tuile). J'ai encapsulé cet algorithme dans une fonction `iswalkable_line()`. Cette fonction génère toutes les coordonnées des tuiles traversées par le regard de l'ennemi vers le joueur. Ensuite, elle vérifie via `iswalkable()` si l'une de ces tuiles est un mur. Si oui, la ligne de vue est brisée, et l'attaque est annulée. De cette manière, les ennemis ne peuvent pas attaquer les joueurs s'il existe un obstacle entre les deux. C'est une mécanique qui rend le jeu tactique et qui permet aussi aux joueurs de se cacher.

2.4.3. Développement des ennemis

Avec le moteur d'IA en place, j'ai pu me concentrer sur la déclinaison des comportements pour offrir des ennemis variés, exigeant chacun une approche différente de la part des joueurs.

2.4.3.1. Le Slasher

Le Slasher est l'ennemi de base du jeu. Armé d'une épée, sa mission est simple : traquer les joueurs jusqu'à être au corps-à-corps pour infliger des dégâts physiques à l'aide de son épée. La difficulté technique résidait dans l'orientation de son attaque. Contrairement à un simple contact qui ferait des dégâts instantanés, je voulais que le coup d'épée balaye une zone spécifique devant lui. J'ai dû concevoir une logique mathématique qui génère un `pygame.Rect` de collision décalé dynamiquement par rapport au centre du Slasher, en fonction de son vecteur directionnel `self.facing`. Si et seulement si la hitbox du joueur entre en intersection avec cette zone générée pendant les frames de l'animation d'attaque, les dégâts sont validés.

2.4.3.2. Le Flinger et ses Orb

Le Flinger est un ennemi qui reste à distance et harcèle les joueurs avec des orbes d'énergie. Son implémentation a nécessité la création d'une architecture parallèle, la classe `Orb`. Bien qu'il s'agisse d'un projectile, j'ai fait le choix architectural de la traiter comme une sous-classe de `Enemy`. Chaque orbe possède sa propre hitbox et ses propres coordonnées de mise à jour. Lors de l'attaque, le Flinger calcule le vecteur directeur entre lui-même et le joueur (v_x, v_y). L'orbe est initialisé avec ce vecteur, une vitesse donnée, et un attribut `life` (une durée de vie). À chaque frame, l'orbe met à jour ses coordonnées. Elle est détruite automatiquement si elle percute un mur (vérifié par `iswalkable()`), si elle touche un joueur, ou si son temps de vie est écoulé, évitant ainsi de surcharger la mémoire avec des projectiles perdus.

2.4.3.3. Le Detonator

Troisième ennemi du jeu, le Detonator adopte l'apparence d'une sorcière. Son comportement de déplacement est similaire au Slasher, mais son modèle d'attaque est radicalement différent. Inspiré de personnages comme Jackie dans le jeu *Brawl Stars*, le Detonator ne frappe pas devant lui : il déclenche une aura toxique circulaire tout autour de lui. Techniquement, lorsqu'il est à portée de tir, le code ne crée pas de rectangle directionnel, mais calcule la distance mathématique ($\text{math.sqrt(dx}^2 + \text{dy}^2)$) entre son centre et tous les joueurs environnants. Si cette distance est inférieure au rayon de l'attaque, les dégâts sont appliqués de manière globale. Cela rend le Detonator particulièrement létal si les deux joueurs sont regroupés, les forçant à se séparer pour l'affronter.

2.4.4. Musique et sons

L'immersion dans un jeu vidéo est indissociable de sa conception sonore. J'ai pris la responsabilité d'intégrer le *Sound Design*, conscient que des mécaniques de jeu abouties paraissent fades sans un retour auditif impactant. Mon travail s'est divisé en deux axes majeurs : les effets sonores (SFX) et la musique.

2.4.4.1. La création et l'intégration des SFX (Sound Effects)

J'ai commencé par établir une nomenclature exhaustive de tous les effets sonores nécessaires au jeu : bruits de pas, coups d'épée, tirs de projectiles, impacts, cris des ennemis, activations de leviers, etc. Pour obtenir des ressources de qualité, j'ai sourcé des éléments libres de droits, principalement via la plateforme *Epidemic Sound*. Cependant, les sons bruts étaient rarement adaptés à un mixage de jeu vidéo. J'ai importé ces *samples* dans mon logiciel de production de musique *FL Studio*. J'ai procédé à de l'égalisation, ajouté de la réverbération, et parfois combiné plusieurs sons pour créer des effets uniques, comme le son de récupération des artefacts.

Côté code, j'ai implémenté ces SFX en utilisant le module `pygame.mixer.Sound`. Pour pallier le problème de répétitivité robotique des sons dans les jeux vidéo, j'ai écrit une logique permettant de stocker des tableaux d'effets sonores. Pour plusieurs attaques par exemple, comme l'attaque à l'épée des joueurs, la fonction `play_random_sound()` sélectionne aléatoirement une variation du son, ce qui rend l'expérience auditive infiniment plus naturelle et agréable.

2.4.4.2. La bande originale et son mastering

Pour la musique de fond, l'objectif était de créer une atmosphère pesante et mystérieuse. J'ai sélectionné plusieurs thèmes musicaux que j'ai à nouveau traités sur *FL Studio*. J'ai réalisé un travail de *rework* et de *re-mastering* : ajustement de la plage dynamique (compression) pour s'assurer que la musique ne masque jamais les indices sonores vitaux du jeu (les SFX de tir ennemi), et création de transitions (*fade in*, *fade out*) pour rendre les transitions de musique plus agréables. Ces pistes ont ensuite été intégrées au moteur principal via `pygame.mixer.music`, avec une gestion paramétrable du volume.

2.4.5. Synchronisation multijoueur des ennemis

Le jeu *Echoes of Lights* étant conçu pour la coopération en réseau (P2P), tout ce que je venais de coder en local devait impérativement être synchronisé entre le joueur hôte et le joueur client.

2.4.5.1. Les actions d'attaque ennemies

Comme la structure réseau de base a été développée par Antoine Lapp, il a fallu que je m'adapte à son système. Nous avons besoin d'objets pour contenir toutes les données d'une attaque afin de pouvoir les reproduire à l'identique chez l'autre joueur. Pour cela, j'ai créé des classes spécifiques héritant de la classe `Action` : `SlasherAttack`, `FlingerAttack`, et `DetonatorAttack`.

Lorsqu'une IA ennemie décide d'attaquer chez l'Hôte, la méthode `attack()` ne déclenche plus les dégâts immédiatement. Elle renvoie un objet `Action` contenant les données nécessaires à sa reproduction. Ainsi, les attaques des ennemis réalisées chez l'hôte sont recrées à l'identique et simultanément chez le client.

2.4.5.2. Partage de l'état et exécution

Cet objet `Action` passe par une méthode `send_to_network()` qui convertit ces paramètres en un dictionnaire JSON. Ce paquet transite vers le client. À la réception, dans la boucle réseau (fichier `network.py`), le code intercepte le message JSON, identifie le type d'attaque, recherche l'ennemi correspondant via son ID dans le `GameContext` local du client, et recrée l'attaque visuellement et logiquement (par exemple, en initialisant un orbe avec exactement la même trajectoire). En parallèle, j'ai ajouté l'envoi continu des états des ennemis (`AI_state`, `hp`, positions cartésiennes) depuis l'hôte pour s'assurer qu'aucun désaccord (désynchronisation) ne se produise si le client subit une légère latence (lag). J'ai réalisé cette partie avec Antoine Lapp, car je n'étais pas encore très à l'aise avec son système de multijoueur. Son assistance m'a permis de mieux comprendre sa logique et de devenir plus autonome dans le développement de cette partie du jeu.

2.4.6. Le boss final

Le boss final représente la dernière étape de mon travail sur ce projet, car il consolide l'ensemble des systèmes développés précédemment : IA, projectiles, multijoueur, son et animation complexe. Ce boss ne se déclenche que lorsque les joueurs ont validé toutes les conditions de victoire (collecte des 4 artefacts) et sont téléportés dans l'arène finale.

2.4.6.1. Les rafales de projectiles

Contrairement aux ennemis standards, le boss est conçu comme une entité colossale et immobile, fonctionnant comme une tourelle crachant des rafales de tirs. Il possède une hitbox plus grande que celles des ennemis normaux et se gère via une variable unique `context.final_boss`, séparée de la liste classique des ennemis.

Pour ses projectiles, j'ai créé une classe de projectile dédiée : `FinalBossProjectile`. À la différence des orbes du `Flinger`, ces projectiles n'ont pas de durée de vie. Ils parcourent l'arène jusqu'à percuter un obstacle ou infliger des dégâts aux joueurs. Ils sont également plus rapides et infligent un peu plus de dégâts.

2.4.6.2. L'aura du boss final

Une des failles évidentes pour les joueurs face à un boss immobile serait de le contourner et de rester cachés derrière lui, dans son angle mort. Pour contrer cette tactique et forcer un affrontement de face en utilisant les couverts de l'arène, j'ai ajouté une "aura" au boss. Dans sa méthode `update()`, le code convertit les coordonnées des joueurs en cartésien et vérifie leur position Y sur la matrice. Si les joueurs s'aventurent "au-dessus" du boss sur l'axe des ordonnées (donc visuellement derrière lui dans la

profondeur isométrique), un système de dégâts s'active en continu (avec un délai `time_before_damage_from_behind` pour infliger des dégâts à intervalles réguliers), afin de leur faire perdre leurs points de vie jusqu'à ce qu'ils meurent.

2.4.6.3. Etats de l'IA et trigonométrie des tirs

Le comportement du boss s'articule autour de trois phases de son IA :

- **IDLE** : une période de répit avec un *cooldown* aléatoire.
- **STUNNED** : si un projectile joueur parvient à le toucher, le boss se désactive momentanément (il n'attaque pas pendant quelques secondes).
- **ATTACK** : la phase la plus technique. Dans cet état, à chaque appel, la fonction d'attaque utilise la trigonométrie pour générer une position de spawn sur un arc de cercle imaginaire devant le boss. En utilisant `math.cos` et `math.sin` associés à un angle aléatoire (`random.uniform`) dans une fourchette définie (ex: 135 degrés d'ouverture), le boss tire une mitraille imprévisible d'orbes. Ce calcul mathématique complexe garantit que chaque projectile s'oriente naturellement vers l'extérieur de cet arc, ce qui permet d'éviter tout croisement des sprites lors de la création du projectile.

2.4.6.4. Le spritesheet particulier

Graphiquement, le boss nécessitait une approche différente. N'ayant pas de déplacement et ne regardant que vers le Sud-Ouest visuel de l'écran, il ne possédait pas les 8 directions des ennemis classiques. Cependant, sa phase d'attaque devait s'animer de manière continue, comme un GIF. J'ai collaboré avec Antoine Muh et Guillaume pour obtenir une animation en boucle fluide. J'ai ensuite réutilisé la fonction `get_animation()` spécifiquement pour le boss afin d'ajuster le multiplicateur d'échelle et le nombre de frames personnalisés.

2.4.6.5. L'attaque du boss et le multijoueur

J'ai créé la classe `FinalBossAttack` pour que l'hôte diffuse les coordonnées trigonométriques précises de chaque projectile généré, tout en intégrant dans le JSON global (dans `network.py`) le partage des variables principales du boss (`AI_state`, `hp`, `cooldowns`). Cette classe permet à l'affrontement final de rester fluide et synchronisé sur les deux machines en même temps.

2.5. Valentin GOUSSOT :

Je suis chargé de la composition des musiques, de la création du sound design ainsi que de la conception du site web du projet Echoes of Lights. Mon objectif principal a été de créer une immersion sonore et visuelle complète afin que les joueurs soient pleinement plongés dans l'univers du jeu dès les premières secondes. Chaque élément audio et visuel a été pensé pour accompagner l'expérience de jeu, renforcer les émotions ressenties par les joueurs et rendre l'univers plus vivant et cohérent. L'ensemble de ce travail a été réalisé au cours des deux premières soutenances du projet, ce qui a permis de mettre en place une identité sonore et visuelle complète dès les premières phases du développement.

2.5.1 Composition musicale

2.5.1 Musique du menu principal

Pour la partie musicale, j'ai composé une partie des thèmes qui accompagnent les joueurs tout au long de leur progression. Chaque morceau a été créé pour correspondre à une situation précise du gameplay et soutenir l'ambiance générale du jeu. La musique joue un rôle important dans l'immersion puisqu'elle permet de transmettre des émotions, de créer une tension ou au contraire d'apporter des moments plus calmes et mystérieux.

Tout d'abord, j'ai réalisé la musique du menu principal. Cette musique a été pensée pour installer immédiatement une ambiance mystérieuse et immersive dès le lancement du jeu. J'ai utilisé des sonorités douces et atmosphériques accompagnées d'une progression harmonique lente afin de créer un sentiment de découverte et de curiosité. La structure du morceau a également été conçue pour être facilement répétable. Cela permet au joueur de rester plusieurs minutes dans le menu sans ressentir de coupure brutale ou de répétition désagréable. Cette musique sert donc à introduire naturellement l'univers du jeu avant même le début d'une partie.

2.5.1 Musiques de combat

J'ai ensuite composé les musiques de combat. Contrairement à celles du menu ou des énigmes, elles ont pour rôle de créer une montée en intensité et de marquer une rupture dans le rythme du jeu. Les combats représentent des moments importants et dynamiques, il était donc essentiel que la musique transmette cette tension. J'ai utilisé des rythmes plus soutenus, des percussions marquées et des sonorités plus puissantes afin d'accentuer les affrontements, notamment lors du combat final qui représente l'un des moments clés du jeu. Malgré cette intensité, les musiques restent cohérentes avec l'univers global de *Echoes of Lights*. Elles accompagnent les actions des joueurs et renforcent les sensations de danger et d'urgence pendant les affrontements. Tous les morceaux ont été pensés pour pouvoir s'adapter à la durée des combats sans devenir répétitifs.

2.5.1 Musiques des salles d'énigmes

J'ai également terminé la composition des musiques destinées aux salles d'énigmes. Ces morceaux ont été créés pour accompagner la réflexion des joueurs sans perturber leur concentration. Pour cela, j'ai travaillé sur des rythmes lents et répétitifs ainsi que sur des sonorités discrètes et atmosphériques. L'objectif était de créer une ambiance calme mais intrigante qui pousse les joueurs à observer leur environnement et à réfléchir aux mécanismes des énigmes. La musique participe directement à l'immersion dans ces phases de jeu et contribue à renforcer la cohérence générale de l'univers.

2.5.2 Sound design

En parallèle de la composition musicale, le sound design a également fait partie de mon rôle dans le projet. J'ai créé et intégré les différents sons liés aux actions des joueurs, aux interactions avec l'environnement, aux énigmes et aux pouvoirs spécifiques présents dans le jeu. Ces effets sonores permettent de rendre le monde plus vivant tout en apportant des informations utiles au gameplay. Certains sons avertissent le joueur d'un danger, confirment qu'une action a été réussie ou signalent l'activation d'un mécanisme. Chaque effet sonore a été conçu pour être clair, reconnaissable et cohérent avec l'ambiance sombre et mystérieuse du jeu.

2.5.2.1 Recherche de ressources sonores

Afin d'améliorer la qualité sonore du projet et d'élargir les possibilités de création, j'ai également effectué plusieurs recherches sur des plateformes spécialisées. J'ai notamment découvert itch.io, une plateforme très utilisée dans le développement indépendant. Elle propose de nombreux packs de sons et de musiques, disponibles gratuitement ou à l'achat. Une grande partie de ces ressources est libre de droit, ce qui permet de les utiliser légalement dans le projet sans problème de licence. Ces packs offrent une grande variété d'effets sonores de qualité qui ont pu être adaptés aux besoins du jeu et intégrés dans différentes situations de gameplay.

2.5.3 Conception du site internet

Enfin, j'ai conçu le site internet de Echoes of Lights. J'ai imaginé ce site comme un espace central permettant de rassembler toutes les informations importantes concernant le projet. Le design du site reprend l'identité visuelle du jeu avec une esthétique sombre et immersive inspirée des thèmes de la lumière et de l'ombre. Dès la page d'accueil, les visiteurs peuvent découvrir une présentation claire du jeu, de son concept et de son gameplay coopératif.

2.5.3.1 Présentation du lore

Le site contient également une section dédiée au lore afin de présenter plus en détail l'univers d'Equinox. Cette partie explique l'histoire du monde, les personnages importants ainsi que les enjeux principaux du scénario. Les informations sont organisées de manière simple et fluide afin de rendre la lecture agréable et compréhensible pour les visiteurs.

2.5.3.2 Présentation technique du projet

Une autre section du site est consacrée aux aspects techniques du projet. Elle présente les différents outils utilisés pendant le développement, les ressources employées ainsi que les liens vers le dépôt GitHub et les rapports d'avancement du projet. Cette partie permet de suivre l'évolution du développement et de mieux comprendre le travail réalisé par l'équipe.

2.5.3.3 Présentation de l'équipe

Enfin, une page spécifique est dédiée à la présentation de l'équipe. Elle met en avant les différents membres du projet ainsi que leurs rôles respectifs afin de montrer l'organisation du travail et la répartition des tâches dans le développement du jeu. Grâce à l'ensemble de ces éléments, le site internet est devenu un véritable support de présentation et de communication autour de Echoes of Lights, tout en restant cohérent avec l'univers immersif du jeu.

2.5.3.4 Page de tracklist des musiques

J'ai également ajouté une page dédiée à la tracklist des musiques du jeu sur le site internet de Echoes of Lights. Cette section permet de regrouper l'ensemble des compositions réalisées pour le projet et de les organiser de manière claire et accessible. Les musiques sont classées par catégories, comme les thèmes du menu principal, les musiques de combat ou encore les ambiances des salles d'énigmes.

Cette organisation permet aux visiteurs de mieux découvrir l'identité sonore du jeu et de comprendre le travail de composition réalisé tout au long du développement. La page sert également de support de présentation pour mettre en avant les différentes ambiances musicales créées pour accompagner les joueurs durant leur progression dans l'univers de Echoes of Lights.

2.5.4 Dictionnaire d'images et correspondance avec les tuiles

La classe `Map` utilise un grand dictionnaire d'images qui associe chaque identifiant numérique de la matrice à une image précise. Cette partie est essentielle, car la matrice ne stocke pas directement des images : elle stocke seulement des nombres.

Par exemple, une case contenant la valeur `1` peut correspondre à une tuile d'herbe, la valeur `150` à un mur en pierre, la valeur `60` à une tuile du puzzle Jeu de la Vie, ou encore les valeurs `90` à `99` aux éléments des énigmes laser.

Le dictionnaire fonctionne donc comme une table de correspondance entre la logique et l'affichage : (un exemple du dictionnaire est présent en annexe 2.5.4.1)

- la matrice stocke des identifiants numériques ;
- le dictionnaire associe ces identifiants à des fichiers image ;
- le moteur affiche l'image correspondant à chaque identifiant.

Cette organisation présente plusieurs avantages. D'abord, elle évite de charger les images à chaque frame : toutes les images sont chargées une seule fois dans le constructeur de la classe `Map`. Ensuite, elle rend le moteur très extensible. Pour ajouter une nouvelle tuile, il suffit d'ajouter une nouvelle couleur dans le pixel art, une fonction `is_xxx(pixel)`, puis une entrée dans le dictionnaire d'images.

Cette séparation entre la matrice et les images permet aussi de modifier facilement l'apparence du jeu sans changer la logique. Si une tuile doit changer graphiquement, il suffit de remplacer le fichier image associé à son identifiant. La logique du moteur reste identique.

La carte est donc générée en deux grandes étapes : d'abord, `image_to_matrix()` transforme l'image pixel art en matrice logique et crée les objets interactifs ; ensuite, la classe `Map` utilise le dictionnaire d'images pour transformer cette matrice en rendu isométrique affiché à l'écran.

Ce système rend la création de niveaux beaucoup plus simple. Au lieu d'écrire manuellement des coordonnées dans le code, il suffit de modifier une image pixel art. La couleur d'un pixel définit non seulement l'apparence de la case, mais aussi son rôle logique dans le jeu.

Sources:

- pixellab.ai
- flat.io
- litch.io
- MuseScore
- Libresprite
- Piskel

3. Récit de la réalisation

3.1. Joies

Antoine Lapp

Malgré ces difficultés qu'on a pu rencontrer, ce projet a également été une grande source de satisfaction. L'un des aspects les plus gratifiants a été de voir le jeu évoluer progressivement d'un simple prototype vers un véritable jeu multijoueur cohérent et fluide. Après avoir passé beaucoup de temps à résoudre des problèmes réseau ou des bugs de synchronisation, réussir à lancer une partie fonctionnelle entre deux joueurs était particulièrement motivant. Le fait d'obtenir un système réseau stable, capable de maintenir la connexion et de synchroniser correctement les actions des joueurs, donne réellement l'impression d'avoir construit quelque chose de concret et techniquement abouti. J'ai aussi apprécié le fait d'avoir réussi à respecter les objectifs que nous nous étions fixés au début du projet. Même si le jeu reste perfectible et qu'il existe encore certainement des bugs ou des optimisations possibles, le résultat final correspond à ce que nous voulions produire : un jeu jouable, agréable et techniquement fonctionnel. Enfin, ce projet m'a aussi apporté une satisfaction plus personnelle, car il m'a permis de progresser dans des domaines nouveaux pour moi, notamment le réseau, le multithreading et le développement collaboratif, tout en découvrant les contraintes réelles d'un projet de développement de jeu vidéo en équipe.

Guillaume Klein

L'un des éléments les plus satisfaisants du projet a été la création du système de génération de carte à partir d'une image pixel art. Au début du développement, les maps étaient construites manuellement dans le code, ce qui rendait leur création lente et difficile à maintenir. Grâce au système RGB, chaque pixel de l'image correspond désormais à un élément du jeu. Le moteur lit automatiquement l'image pour générer la matrice complète du monde, ce qui a énormément simplifié la création des niveaux. Modifier une salle revient simplement à modifier une image, ce qui a rendu le level design beaucoup plus rapide et intuitif. La modularité des énigmes a également été très satisfaisante. En combinant la lecture RGB, la génération dynamique de la matrice et le GameRegistry, il est devenu très simple d'ajouter de nouveaux contenus au jeu. Un simple changement de couleur dans l'image permet désormais d'ajouter : des leviers ; des miroirs ; des ennemis ; des lasers ; des puzzles ; des tuiles du jeu de la vie ; ou encore de nouvelles salles entières.

Le système utilise notamment la composante bleue des pixels comme identifiant de groupe afin de relier automatiquement plusieurs objets entre eux. Cette architecture a énormément facilité l'extension des différentes branches du jeu et a rendu le moteur très flexible. Le projet est progressivement devenu une sorte d'éditeur piloté par image, où la création de contenu se fait presque directement dans le pixel art de la map.

Antoine Muh

Le projet *Echoes Of Lights* a apporté de nombreuses satisfactions créatives et techniques. La création des personnages principaux, Aeden et Lyra, a été particulièrement gratifiante, car elle a permis de donner vie à des personnages uniques grâce au pixel art et aux animations. La conception des ennemis variés du bestiaire a aussi été une expérience enrichissante sur le plan artistique. La stabilisation du menu multijoueur a constitué une grande réussite technique : voir une interface fluide, intuitive et fonctionnelle après de nombreuses corrections a été très satisfaisant. Le projet a également permis d'apprendre et de progresser dans plusieurs outils et technologies comme Python, Pygame et différents logiciels de pixel art.

Le travail en équipe a joué un rôle important, grâce à une bonne collaboration entre les membres du groupe et à l'intégration des différentes compétences. Enfin, voir la vision artistique du jeu se construire progressivement et comprendre concrètement le fonctionnement d'un projet de développement de jeu vidéo ont renforcé la motivation et la passion pour ce domaine.

Gustave Schwien

Ayant beaucoup travaillé sur la conception des ennemis et de leur IA, l'une de mes principales joies au cours du projet a été de voir mes ennemis prendre vie lors de leur implémentation en jeu, grâce à leur IA qui se comporte différemment en fonction de sa distance par rapport au joueur et à l'environnement. Le fait de passer du code théorique et redondant au visuel de l'ennemi qui fonctionne fait partie de mes plus grandes satisfactions sur ce projet, en sachant que je n'avais jamais autant programmé sur un seul projet (qui s'avère d'ailleurs relativement complexe).

L'algorithme A* est également une fierté pour moi dans ce projet. En effet, j'ai eu beaucoup de mal à l'optimiser et à le rendre fonctionnel, et aujourd'hui je suis très satisfait du résultat. Mon temps de débogage en a valu la peine.

Ensuite, j'ai développé le boss final quasiment seul, multijoueur compris. Au début du projet, j'étais assez perdu dans le fonctionnement du multijoueur ; et finalement grâce aux explications d'Antoine Lapp et de Guillaume en plus de ma compréhension personnelle j'ai réussi à intégrer sa logique, ce qui m'a permis d'implémenter la partie multijoueur du boss final en toute autonomie.

Enfin, je suis assez satisfait de mon travail sur la musique et les *SFX* (effets audio) car, bien que passionné et producteur de musique à mes heures perdues, je n'avais jamais créé de bande originale, et encore moins pour un jeu style dark fantasy. De plus, je n'avais jamais réalisé d'effets audio non plus.

Globalement, bien qu'ils ne soient pas parfaits, je trouve que la musique et les sons de notre jeu sont adaptés à son thème et créent une vraie atmosphère dark fantasy. Ils sont à leur place.

Valentin Goussot

Ce projet m'a permis de développer mes compétences dans plusieurs domaines qui me passionnent, notamment la composition musicale, le sound design et la création web. J'ai particulièrement apprécié le fait de pouvoir participer à la création de l'univers du jeu et de contribuer à l'immersion des joueurs grâce au travail sonore. Composer des musiques adaptées aux différentes situations du gameplay et créer des effets sonores cohérents avec l'ambiance du jeu a été une expérience très enrichissante. J'ai également aimé travailler en équipe et voir le projet évoluer au fil des soutenances. Enfin, la conception du site internet m'a permis de mettre en valeur le projet et de créer un espace clair et immersif pour présenter notre travail.

3.2. Peines

Antoine Lapp

Les principales difficultés de ce projet ont été liées à la complexité du multijoueur et au travail collaboratif. L'aspect réseau a représenté un véritable défi technique, notamment à cause de la latence et de la synchronisation entre les deux joueurs. Même lorsque la connexion fonctionnait, il fallait constamment vérifier que les informations envoyées et reçues restaient cohérentes afin d'éviter des décalages entre les écrans des joueurs. Cela demandait de réfléchir à la fréquence d'envoi des données, à l'ordre dans lequel elles étaient traitées, mais aussi à la manière de garder un jeu fluide malgré les échanges réseau permanents. Certains problèmes étaient également plus imprévus, comme l'impossibilité d'écrire

directement dans le dossier %APPDATA% pour gérer les sauvegardes, ce qui nous a obligé à chercher des alternatives et à adapter une partie du système de fichiers du jeu. En parallèle, la collaboration sur le code a été une autre source de difficulté. C'était la première fois que je travaillais sur un projet aussi important avec plusieurs personnes modifiant le même dépôt Git. Nous avons donc rencontré de nombreux merge conflicts, parfois compliqués à résoudre, surtout lorsque plusieurs parties du projet dépendaient les unes des autres. Cela m'a appris qu'un projet en équipe ne consiste pas seulement à coder, mais aussi à communiquer, organiser le travail et anticiper les problèmes d'intégration.

Guillaume Klein

L'une des principales difficultés du projet a été la mise en place du multijoueur, notamment pour la gestion des énigmes et des interactions synchronisées. Au début du développement, certaines actions étaient exécutées directement dans les objets ou dans la boucle du joueur, ce qui fonctionnait en local mais provoquait rapidement des incohérences en réseau. Par exemple, une porte pouvait s'ouvrir chez un joueur mais rester fermée chez l'autre. La difficulté venait du fait qu'une simple interaction pouvait entraîner plusieurs conséquences : modification de la matrice, ouverture de portes, propagation entre leviers, lecture de sons ou encore déclenchement d'animations.

La création du système d'actions a finalement permis de résoudre ces problèmes en centralisant toutes les interactions du jeu sous une même architecture. Cela a grandement simplifié la synchronisation des énigmes et stabilisé le fonctionnement du multijoueur.

Les déplacements diagonaux dans le moteur isométrique ont également été compliqués à mettre en place. Les premières versions utilisaient des calculs beaucoup plus complexes afin d'essayer de suivre correctement les lignes de la grille isométrique. Cela entraînait des déplacements peu naturels et des collisions parfois imprécises. Après plusieurs essais, un ratio beaucoup plus simple a été trouvé pour les diagonales, ce qui a permis d'obtenir des mouvements fluides et cohérents visuellement tout en simplifiant fortement le code.

La gestion des différentes boucles du jeu a aussi représenté une difficulté importante vers la fin du projet. Le moteur devait désormais gérer le gameplay, les transitions, les cinématiques, le choix final et les crédits sans casser l'affichage principal. Comme le projet utilise un écran logique redimensionné ensuite à l'écran réel, certaines transitions provoquaient des problèmes visuels ou des éléments mal positionnés. Il a donc fallu réorganiser le pipeline d'affichage afin de garder un rendu stable entre toutes les phases du jeu.

Antoine Muh

Le développement d'*Echoes Of Lights* a aussi été marqué par de nombreuses difficultés. Les premières versions du menu principal étaient remplies de bugs et de problèmes techniques, ce qui a demandé beaucoup de corrections et de patience. L'absence de champ de texte intégré dans Pygame a compliqué la création du système de connexion multijoueur. Le pixel art isométrique a représenté un défi majeur : chaque personnage devait être animé dans plusieurs directions avec de nombreuses frames, ce qui demandait énormément de temps et de précision. Certains éléments, comme la grande épée du Slasher, ont nécessité de multiples essais avant d'obtenir un résultat satisfaisant. L'intégration des animations dans le moteur du jeu a aussi provoqué des problèmes techniques fréquents, obligeant à modifier continuellement les créations graphiques. La double responsabilité entre programmation et graphisme a parfois créé une forte pression et une charge de travail difficile à gérer. Malgré cela, ces difficultés ont permis de développer la persévérance, les compétences techniques et l'expérience du travail en équipe.

Gustave Schwien

J'ai rencontré des difficultés tout au long du projet car je n'avais jamais réalisé de projet de programmation aussi étendu auparavant.

Ma première difficulté a porté sur les hitboxes : je comptais à ce que les hitboxes prennent la forme de l'image des entités, mais en isométrique ce n'était pas la bonne démarche. En effet, comme le jeu est originalement en 2D, la hitbox des joueurs doit prendre la forme de la base de l'entité, et non pas sa hauteur. Par conséquent, j'ai dû modifier la forme des hitboxes déjà existantes et revoir la globalité des calculs prenant en compte leurs coordonnées comme elles ont été modifiées lors du changement de forme. Ensuite, j'ai eu beaucoup de mal à produire un algorithme de pathfinding efficace et performant, sans bugs, et peu gourmand en puissance de calcul. En effet, au début, les ennemis mettaient à jour leur chemin à chaque frame, ce qui demandait des milliers de calculs par seconde : ce n'était pas du tout optimisé. Avant cela, je n'arrivais pas à cerner comment un programme pouvait demander assez de calculs à un processeur pour le faire ramer. Grâce à cet algorithme, je m'en suis rendu compte.

De plus, j'ai rencontré de nombreux bugs avec l'algorithme A*, et en général ils entraînaient toujours un crash. Par exemple, parfois, l'algorithme détectait l'ennemi ou le joueur sur une tuile sur laquelle ils ne sont pas censés pouvoir marcher. Par conséquent, l'algorithme cherchait un moyen d'atteindre la tuile en évitant la tuile, ce qui est évidemment impossible. Cela entraînait des calculs infinis qui faisaient crash le jeu. Ce bug était lié à un décalage de la position des joueurs et ennemis par rapport à leurs coordonnées réelles, qui m'a pris du temps à résoudre. Ce bug m'a aussi fait ouvrir les yeux sur le fait que mon algorithme n'était pas optimisé, et qu'il fallait que je l'améliore. Seulement, je ne voyais absolument pas comment. L'une des solutions que j'ai trouvées pour limiter les calculs inutiles a été de limiter le nombre de calculs pour une recherche de chemin. En effet, plus le joueur et l'ennemi sont éloignés, plus la recherche de chemin sera lente pour l'ennemi. Cela a permis de panser légèrement la latence causée par l'algorithme. Finalement, j'ai simplement ajouté une limite de distance simple à partir de laquelle l'ennemi ne détecte plus le joueur et se met en IDLE.

Le dernier "gros" problème que j'ai rencontré avec l'algorithme A* a été le fait que l'ennemi pouvait se retrouver trop loin du joueur pour l'attaquer, même en étant sur la même tuile que lui. Par conséquent, une fois tout son chemin parcouru, il se figeait et ne pouvait pas attaquer. J'ai passé beaucoup de temps sur ce problème car je ne savais pas comment le résoudre sans en créer un autre. La solution que j'ai trouvée a été d'ajouter une autre limite de distance par rapport au joueur, qui permettait cette fois à l'ennemi de se déplacer sans algorithme A* mais simplement à l'aide de vecteurs directeurs en prenant pour cible la hitbox du joueur, s'il était assez proche de lui (quelques dizaines de pixels). Il a fallu trouver une valeur qui évite aux ennemis de rester coincés tout en évitant de leur permettre de traverser les murs qui les séparent s'ils sont chacun d'un côté.

J'ai ensuite rencontré d'autres problèmes relativement proches de l'algorithme de pathfinding, notamment avec la fonction `iswalkable_line()` qui utilise un algorithme de Bresenham. En effet, j'ai peiné à adapter l'algorithme au jeu : j'ai longtemps cru qu'il fonctionnait correctement malgré des crashes causés par `iswalkable_line()`, alors que sa logique de base était en réalité incorrecte. J'ai passé beaucoup de temps à corriger ce bug car je me trompais sur la source du problème.

En outre, j'ai fait face à des bugs particulièrement embêtants avec les orbes du Flinger. Pour que les orbes soient synchronisés entre les deux parties des joueurs, il faut qu'ils soient ajoutés dans une liste de l'objet `GameContext` du jeu (qui contient toutes les informations partagées à la partie network du jeu). Or, jusqu'à maintenant nous n'avions de projectiles que pour les joueurs (qui ont été développés en parallèle par mes camarades). Ma première intention a été d'ajouter les projectiles ennemis dans cette liste déjà existante. Cependant, j'ai rapidement été confronté à un problème majeur : le comportement des projectiles ennemis et joueurs sont complètement différents, et cela notamment à cause de leur cible : les projectiles ennemis infligent des dégâts aux joueurs, et les projectiles joueurs infligent des dégâts aux ennemis. J'ai donc été obligé de créer une nouvelle liste `enemy_projectiles` pour les séparer des joueurs. Cela impliquait également de recréer les fonctions auxiliaires aux projectiles joueurs pour les projectiles ennemis, comme

la fonction `check_collision` vérifiant si le projectile a touché un joueur, ou `draw_projectile` pour les afficher à l'écran. Par souci de simplicité, je n'ai pas recréé de nouvelle fonction mais simplement intégré leur comportement aux endroits où je les appelais pour les joueurs.

Par ailleurs, un autre problème récurrent lors du développement des orbes a été les décalages entre l'image de l'orbe à l'écran, sa hitbox, sa position réelle à l'écran, ses coordonnées théoriques sur la fenêtre Pygame et ses coordonnées isométriques. Il a été particulièrement difficile pour moi de me repérer dans les différents systèmes de coordonnées, et de cerner l'origine des différents décalages lors de l'affichage à l'écran. Par conséquent, il est encore possible d'observer de légers bugs d'affichage, bien que j'aie fait tout mon possible pour les corriger au maximum.

Valentin Goussot

Certaines parties du projet ont été plus compliquées à réaliser, notamment la création d'une ambiance sonore cohérente pour l'ensemble du jeu. Il a fallu composer des musiques différentes selon les situations tout en gardant une identité commune afin que l'univers reste immersif. Trouver le bon équilibre entre des musiques présentes et des sons qui ne gênent pas le gameplay a également demandé plusieurs essais. Le sound design a aussi représenté une difficulté, car chaque effet sonore devait être clair, reconnaissable et adapté aux actions des joueurs. Enfin, la création du site internet a demandé du temps pour organiser correctement les informations et réaliser un design qui corresponde à l'ambiance sombre et immersive de *Echoes of Lights*.

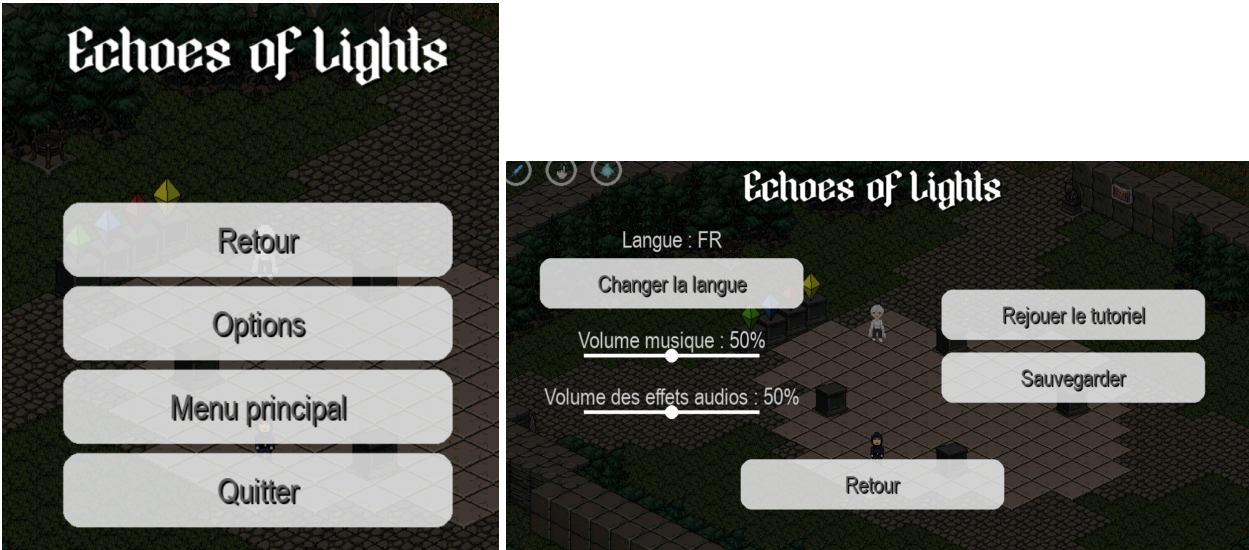
4. Annexes

Composant	Soutenance 1	Soutenance 2	Soutenance Final
Pixel Art	20%	50%	100%
Musique	0%	50%	100%
Animation	0%	50%	100%
Sound Design	0%	50%	100%
Multijoueur	90%	100%	100%
Gameplay	50%	90%	100%
Map	20%	50%	100%
Déroulement Jeu	80%	100%	100%
Affichage Jeu	80%	100%	100%
Dev. Joueurs	70%	90%	100%
Affichage Menu	30%	100%	100%
IA	30%	70%	100%

1.2.1 Tableau des avancements des différents domaines du jeu

```
JSON [copy] [share] [more]
{"sdp": "v=0\r\no=- 3970739909 3970739909 IN IP4 0.0.0.0\r\ns=-\r\nnt=0 0\r\na=group:BUNDL  
E 0\r\na=msid-semantic:WMS *\r\nm=application 59947 UDP/DTLS/SCTP webrtc-datachannel\r\nnc  
=IN IP4 10.2.0.2\r\na=mid:0\r\na=sctp-port:5000\r\na=max-message-size:65536\r\na=candidat  
e:d6ee6c8481721c8f410c5e2bcc4334f8 1 udp 2130706431 10.2.0.2 59947 typ host\r\na=candidat  
e:ae5c2359578edcf27f2d217b587052e6 1 udp 2130706431 192.168.56.1 59948 typ host\r\na=cand  
idate:a7f0c3a094b0dc7d1343263db553d60c 1 udp 2130706431 192.168.1.6 59949 typ host\r\na=c  
andidate:36780b3c8d48149a320c24d2c2fb881c 1 udp 1694498815 190.2.153.209 61566 typ srflx  
raddr 10.2.0.2 rport 59947\r\na=end-of-candidates\r\na=ice-ufrag:s9co\r\na=ice-pwd:1DFpIF  
c6F8YpLVN13cb2KD\r\na=fingerprint:sha-256 AC:4E:E2:7C:3D:82:EB:2D:3F:40:12:AE:62:00:20:6  
C:93:73:CB:1C:C8:69:64:3B:6F:11:C0:6D:95:4B:DF:96\r\na=fingerprint:sha-384 89:35:52:36:B  
5:33:5D:68:3D:6C:28:F5:70:0C:50:15:6E:36:34:AA:C1:9F:37:CE:38:1F:03:A9:38:6B:C7:45:70:83:  
10:92:0B:8F:77:41:B6:FD:89:CD:F3:80:38:26\r\na=fingerprint:sha-512 98:C2:AD:68:43:73:D5:C  
0:AA:97:07:31:23:22:B2:E1:97:BB:3E:4E:07:A6:C7:66:85:C7:EE:1C:91:A8:AB:93:A7:48:BA:21:ED:  
95:71:C4:2A:20:54:AB:FB:15:1B:41:99:DB:80:1C:A4:08:6C:3B:0E:5C:A1:79:2E:3F:C1:E7\r\na=set  
up:active\r\n", "type": "answer"}
```

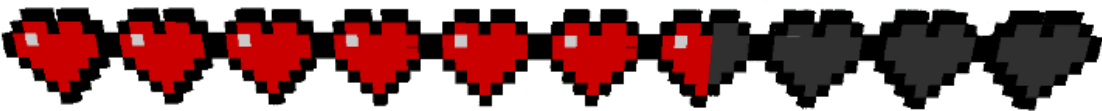
2.1.2.1 Une clé SDP permettant la connexion P2P



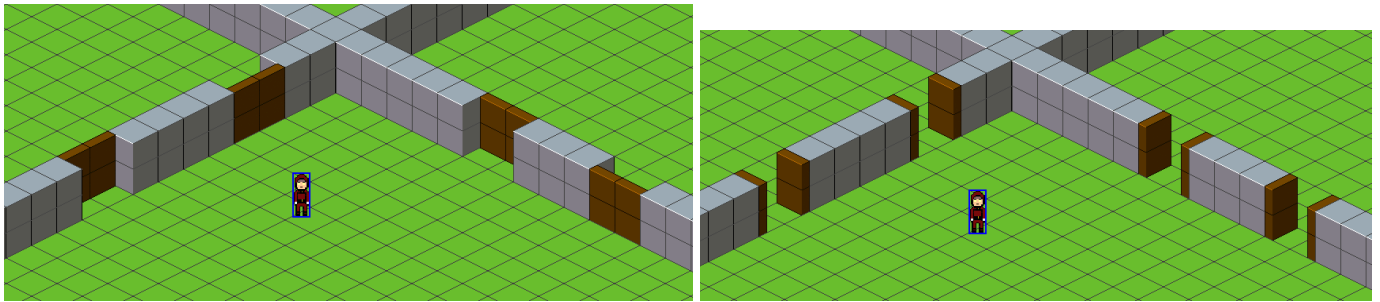
2.1.3.1 Le menu pause



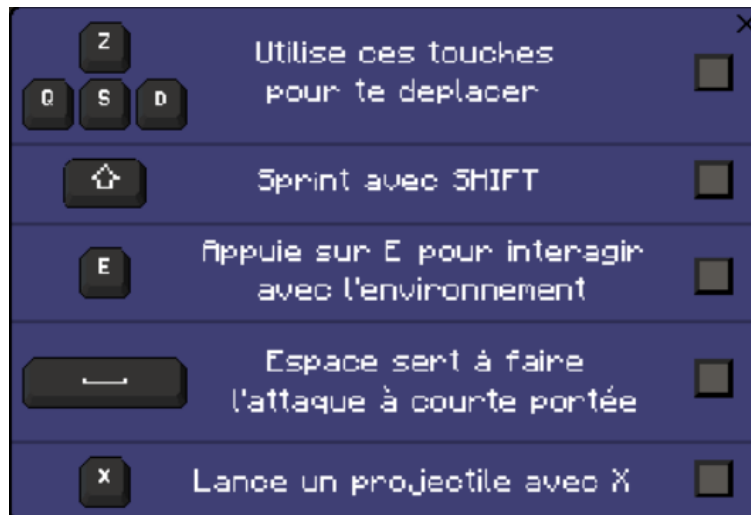
2.1.3.2 HUD avec barre de vie et cooldown



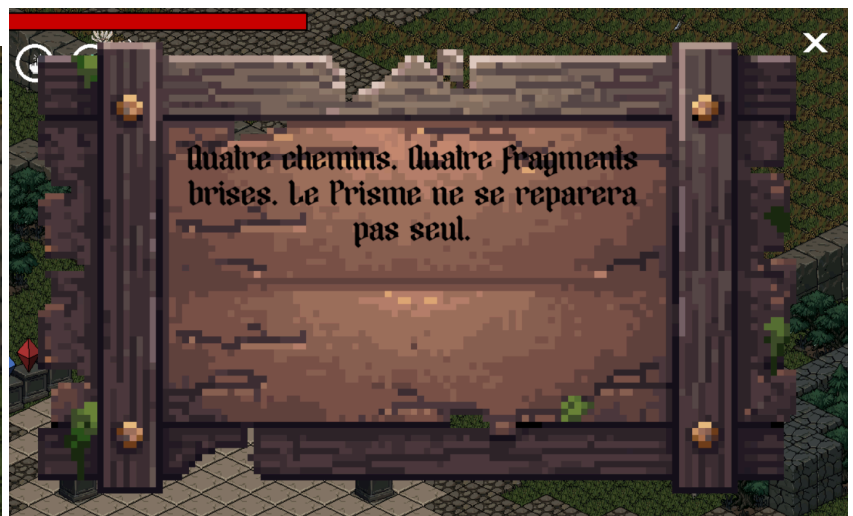
2.1.3.3 Barre de vie du jeu Minecraft



2.1.4.1 Le système de portes pour les énigmes (lors de tests)



2.1.5.1 Le tutoriel de l'assignation des touches sous forme de check-list



2.1.5.2 Les panneaux sur la carte et en plein écran


```

Echoes_Of_Lights
├── assets                                # Tous les assets comme avant, reste inchangé
│   └── ...
├── core                                # Le cœur du jeu : logique et fonctionnement global
│   ├── __pycache__
│   ├── __init__.py
│   ├── actions.py
│   ├── audio.py
│   ├── game_context.py
│   ├── game_logic.py
│   ├── gameStateRegistry.py
│   ├── save_manager.py
│   └── interact.py
├── entities                            # Ce qui est relatif aux entités / IA
│   ├── __pycache__
│   ├── __init__.py
│   ├── entity.py
│   ├── enemy.py
│   ├── pathfinding.py
│   └── player.py
├── network                             # Le réseau
│   ├── __pycache__
│   ├── __init__.py
│   └── network.py
├── rendering                           # L'affichage, l'UI, les menus
│   ├── __pycache__
│   ├── __init__.py
│   ├── animations.py
│   ├── isometric_motor.py
│   ├── menu_components.py
│   └── menu.py
├── .gitignore
├── requirements.txt
├── requirements_dev.txt
├── README.md
└── echoes_of_lights.py                # Fichier de lancement du jeu

```

2.1.6.1 L'architecture finale du projet

Au coeur du sanctuaire se dressent quatre voies oubliées,
chacune abritant un artefact ancien imprégné de lumière et d'ombre.

Votre objectif sera d'explorer ces branches et de récupérer les artefacts
en interagissant avec les vestiges qui jalonnent votre chemin.

Mais chaque passage mettra vos capacités à l'épreuve :

- des énigmes environnementales où l'observation et l'interaction seront essentielles pour progresser
- des créatures hostiles qu'il faudra affronter en maîtrisant vos deux compétences d'attaque.

2.2.8 Une capture d'écran de la fenêtre d'introduction



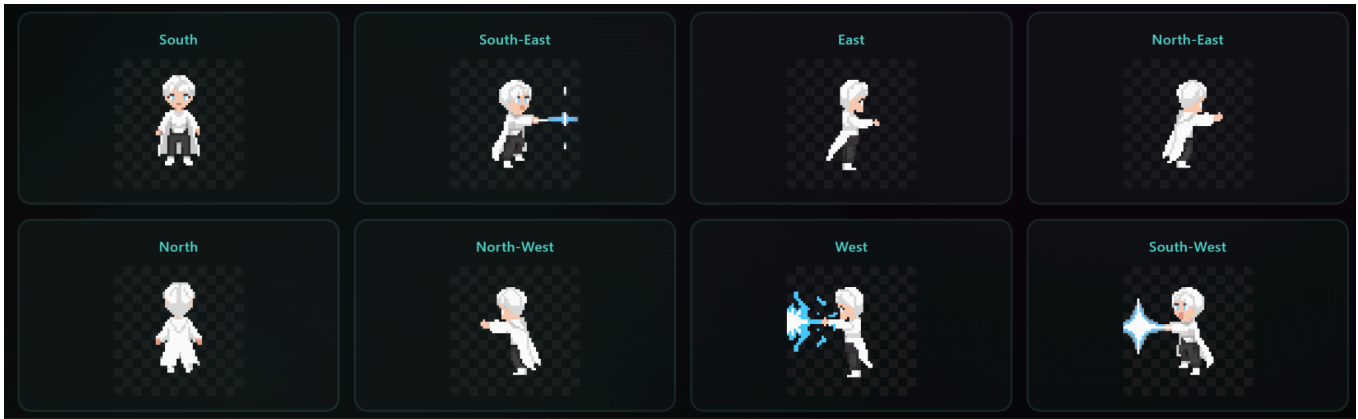
2.2.9. Une capture d'écran du dilemme final



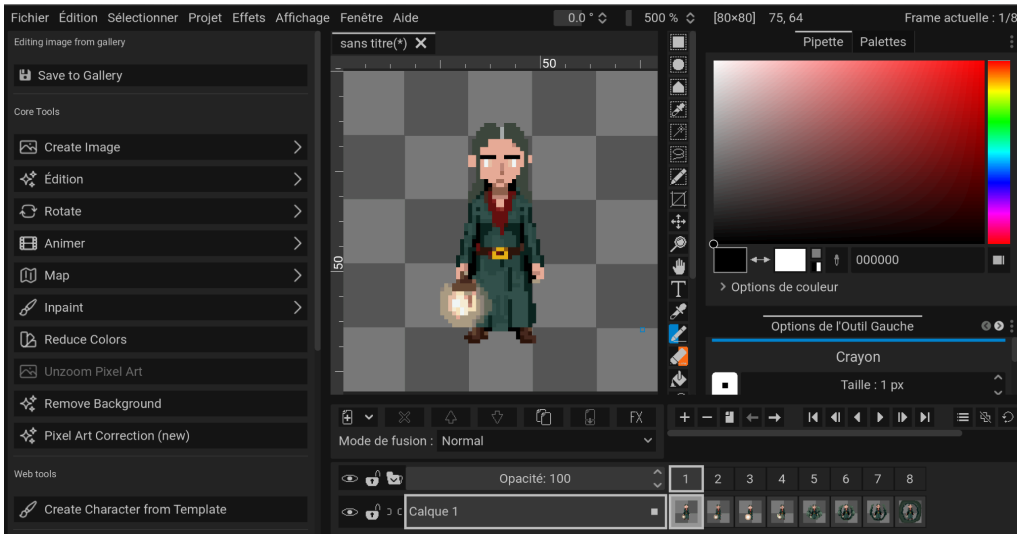
2.3.1 Première version du menu



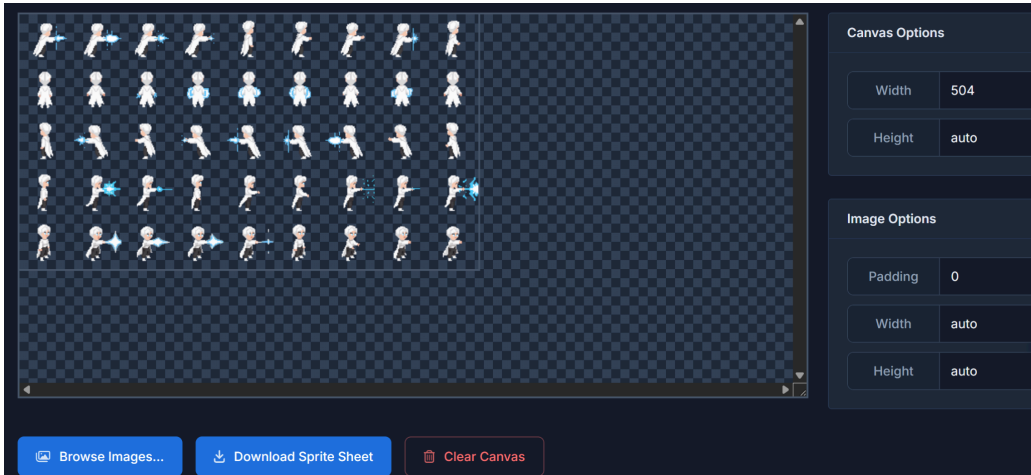
2.3.2 Version finale du menu



2.3.3 : Affichage dans Pixellab.ai



2.3.4 : Edition de pixel art dans Pixelorama



2.3.5 Création de spritesheet dans codeshack.io



2.3.6 Aeden



2.3.7 Lyra



2.3.8 : Exemple de sprite sheet (ici animation de marche d'Aeden)



2.3.9 Flinger



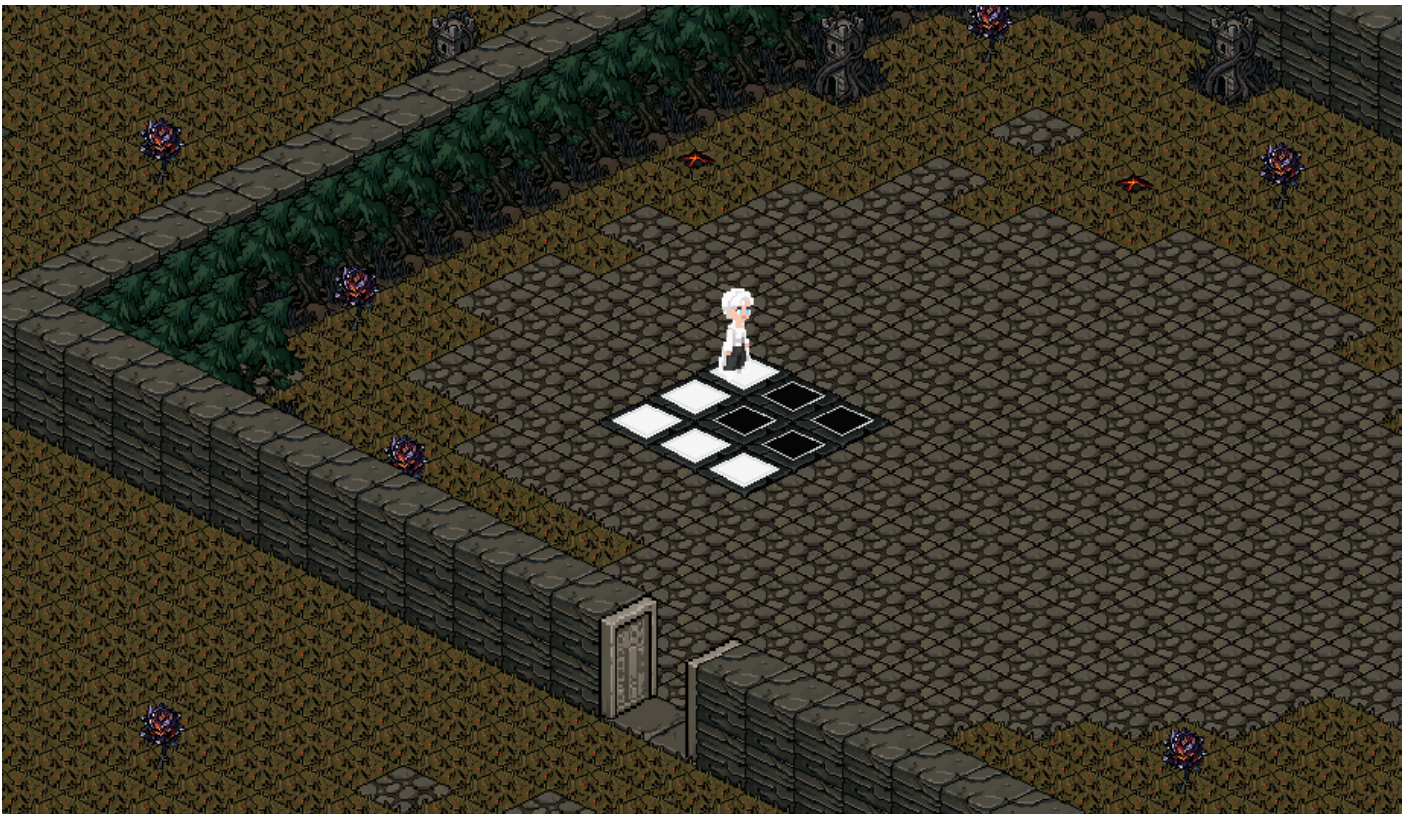
2.3.10 Slasher



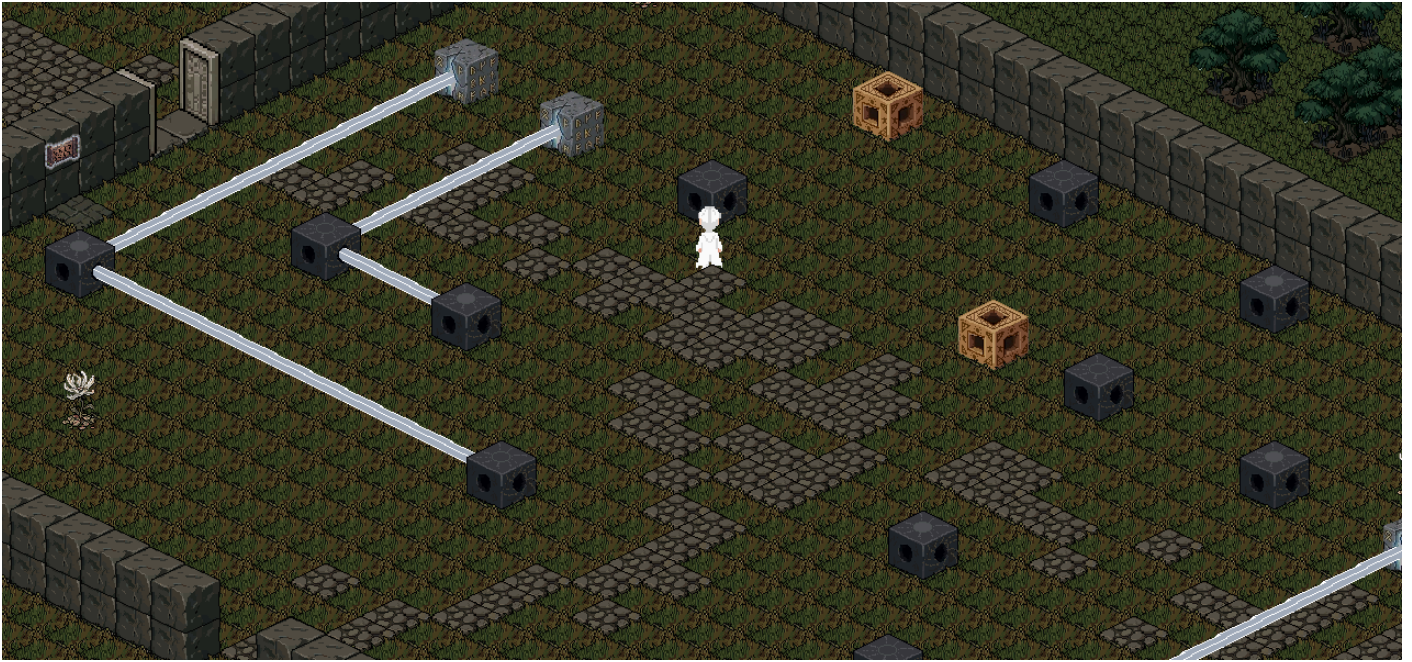
2.3.11 Detonator



2.4.1.2. Exemple d'énigme des leviers



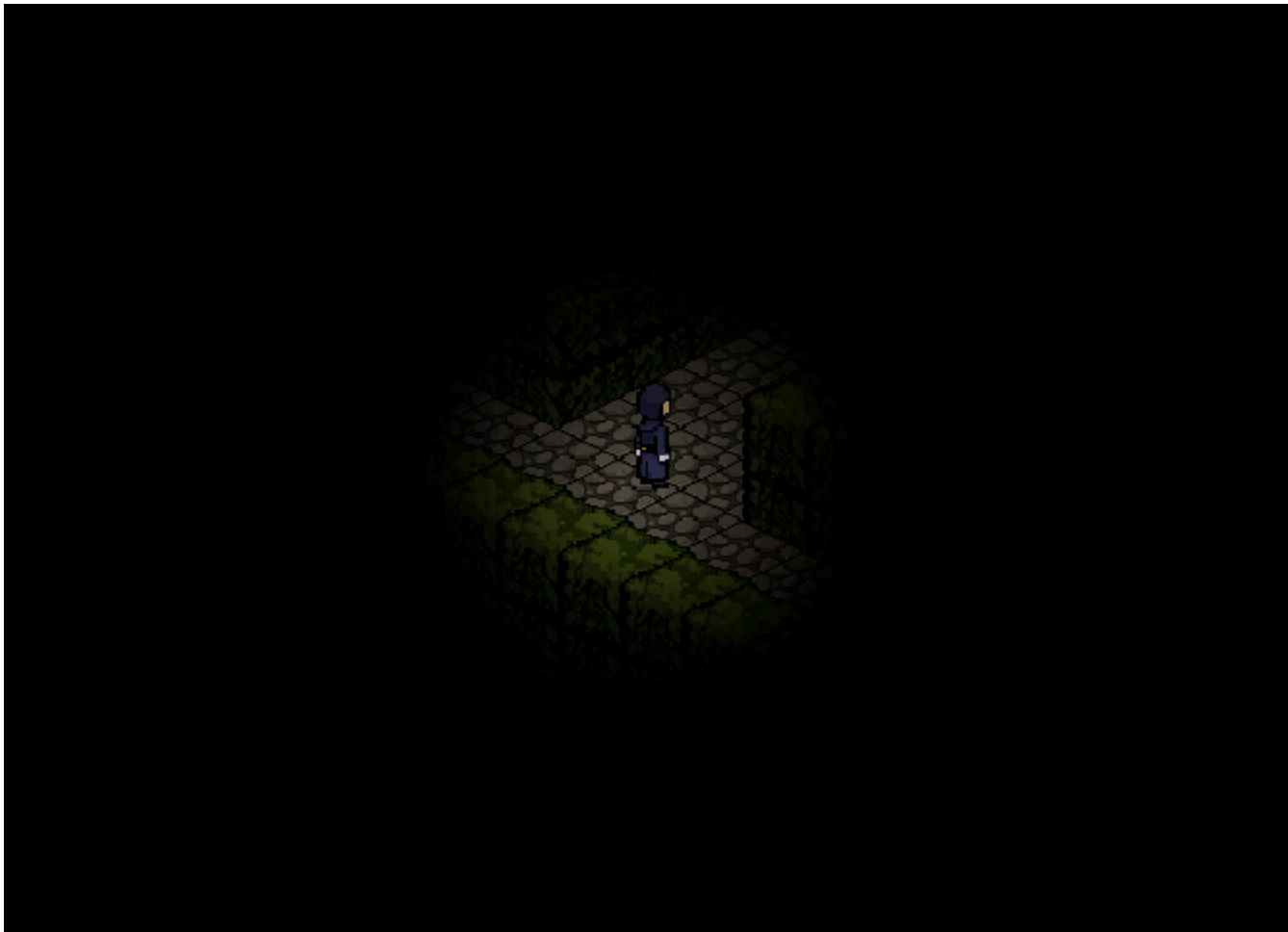
2.4.1.2. Exemple d'énigme des tuiles



2.4.1.2. Exemple d'énigme des lasers



2.4.1.2. Exemple de labyrinthe (point de vue d'Aeden)



2.4.1.2. Exemple de labyrinthe (point de vue de Lyra)



2.4.3. Gameplay d'Aeden attaqué par un Slasher, un Flinger et un Detonator



2.4.6. Le combat face au boss final et ses rafales de projectiles

```
# Les images
self.images = {
    "tiles": {
        # grass
        1: pygame.image.load(resource_path("assets/images/game/grass/simplegras:
        4: pygame.image.load(resource_path("assets/images/game/grass/cube_64_fil

    # énigmes
    9: pygame.image.load(resource_path("assets/images/game/tileset/enemy_spa
    10: pygame.image.load(resource_path("assets/images/game/enigmes/levier_h
    11: pygame.image.load(resource_path("assets/images/game/enigmes/levier_l
    12: pygame.image.load(resource_path("assets/images/game/enigmes/levier_h
    13: pygame.image.load(resource_path("assets/images/game/enigmes/levier_l
    30: pygame.image.load(resource_path("assets/images/game/enigmes/artefact
    31: pygame.image.load(resource_path("assets/images/game/enigmes/artefact
    32: pygame.image.load(resource_path("assets/images/game/enigmes/artefact
    33: pygame.image.load(resource_path("assets/images/game/enigmes/artefact
    40: pygame.image.load(resource_path("assets/images/game/enigmes/panneau_l
    41: pygame.image.load(resource_path("assets/images/game/enigmes/panneau_l
    42: pygame.image.load(resource_path("assets/images/game/enigmes/panneau_l
    43: pygame.image.load(resource_path("assets/images/game/enigmes/panneau_l
    60: pygame.image.load(resource_path("assets/images/game/enigmes/LifeGame
    61: pygame.image.load(resource_path("assets/images/game/enigmes/LifeGame
    90: pygame.image.load(resource_path("assets/images/game/enigmes/mirror1.
    91: pygame.image.load(resource_path("assets/images/game/enigmes/mirror2.

    }
}
```

tionnaire permettant de charger les tiles